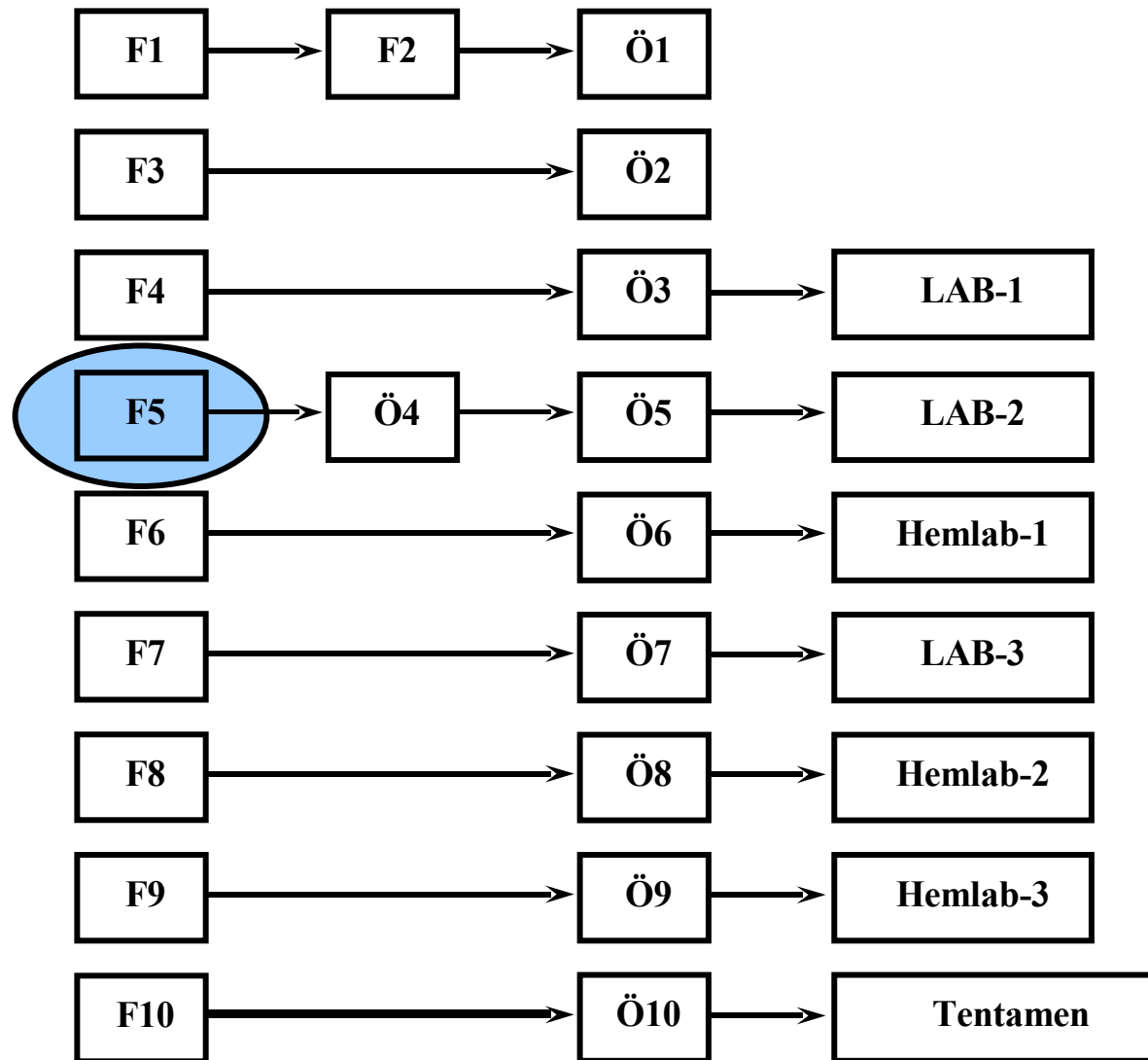


2G1518 Datorteknik

Föreläsning 5, våren 2007

Bussar
In- och utmatning – I/O
Pollning
Handskakning

2G1518 Datorteknik



Assemblerkod
4 stegs pipeline

Nios2time

Nios2io

C-kod

Nios2int

Cache-minnen

CPU-scheduling

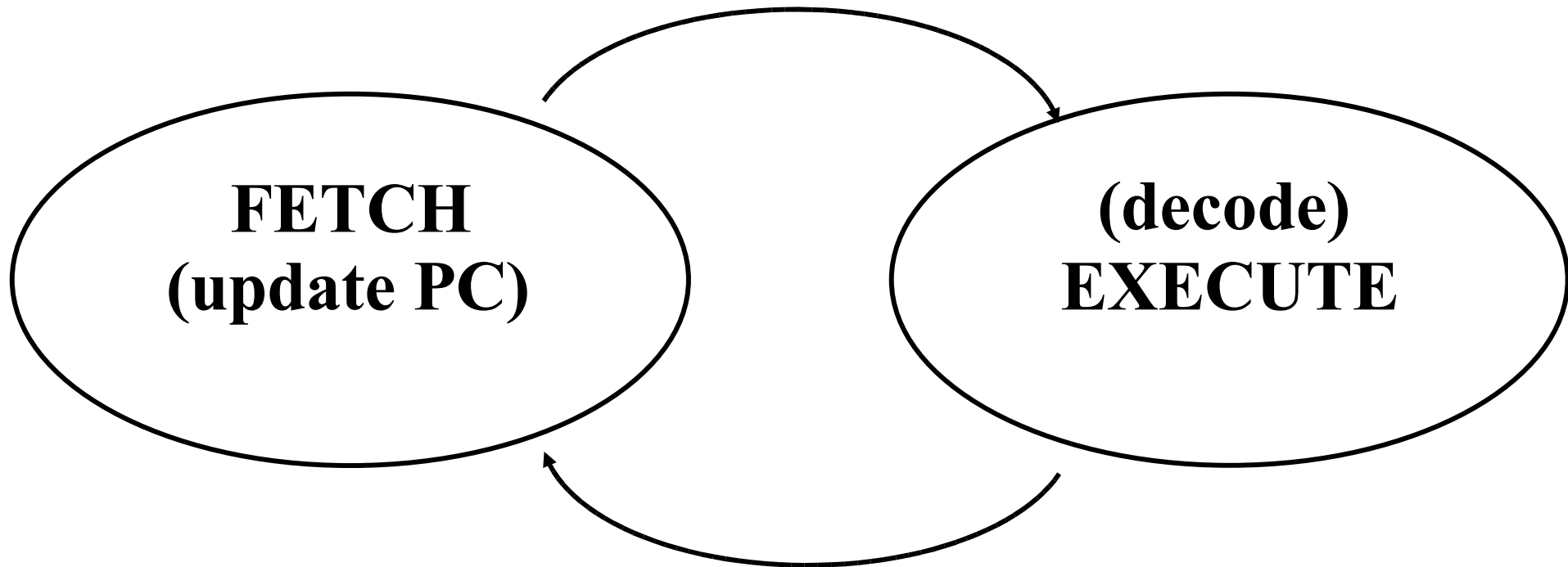
Rubriker

- Bussöverföring – läsning och skrivning
- Adressering – memory-mapped, I/O-mapped
- Parallellport – uppbyggnad och funktion
- Handskakning
- Serieport – funktion, paritetskontroll
- Eko-program, kommunikationsprogram
- Timer

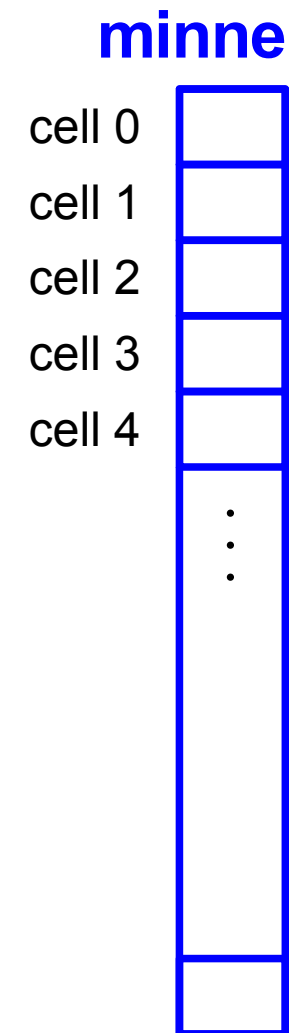
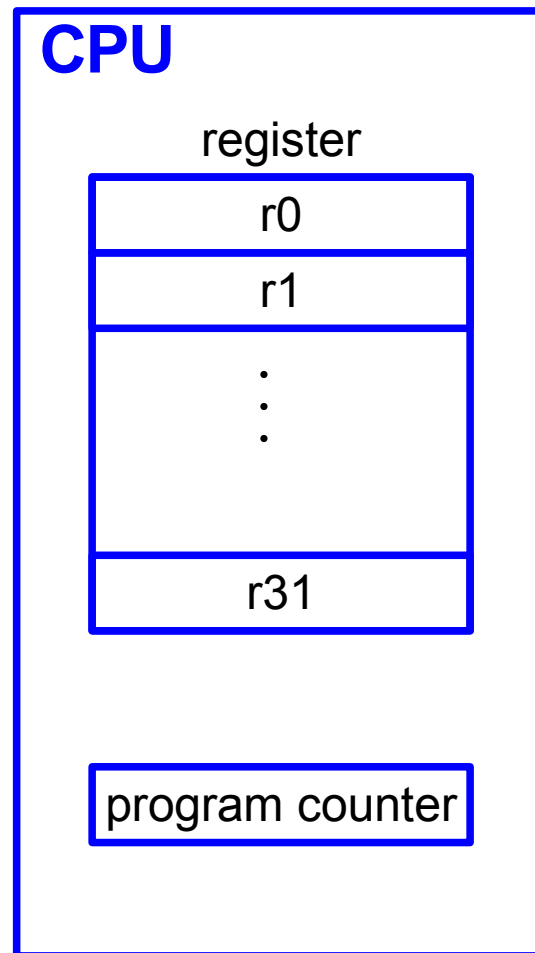
Att läsa

- Kursboken, avsnitt:
 - 1.4 Maskinvara
 - 2.6 Bussar och enkel I/O-hantering
 - 5.2 Koppling till omvärlden, sida 151–156
 - 9.4 Kommunikation mellan processor och I/O-enheter

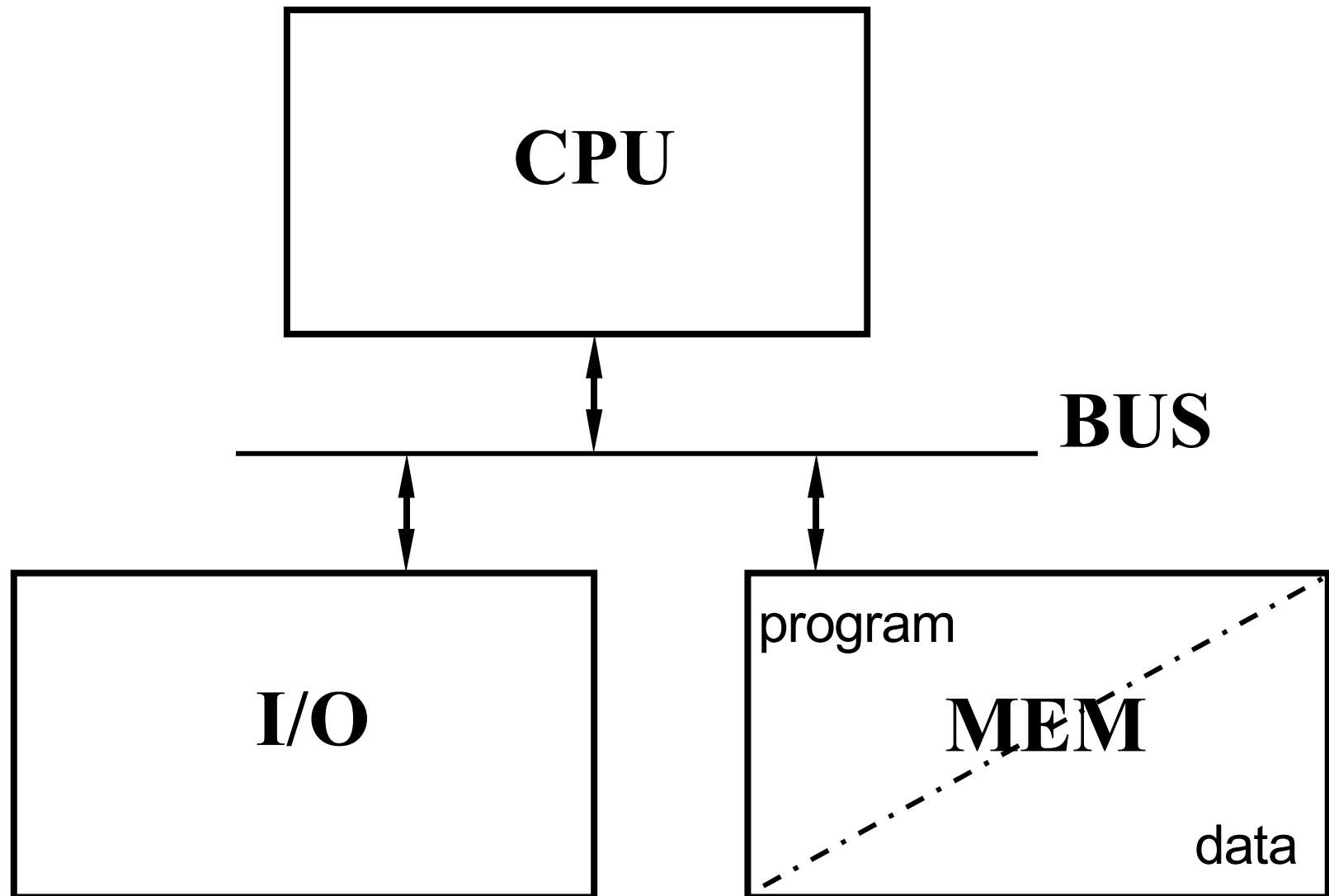
Programsekvensering i två steg



Programmerarens datormodell



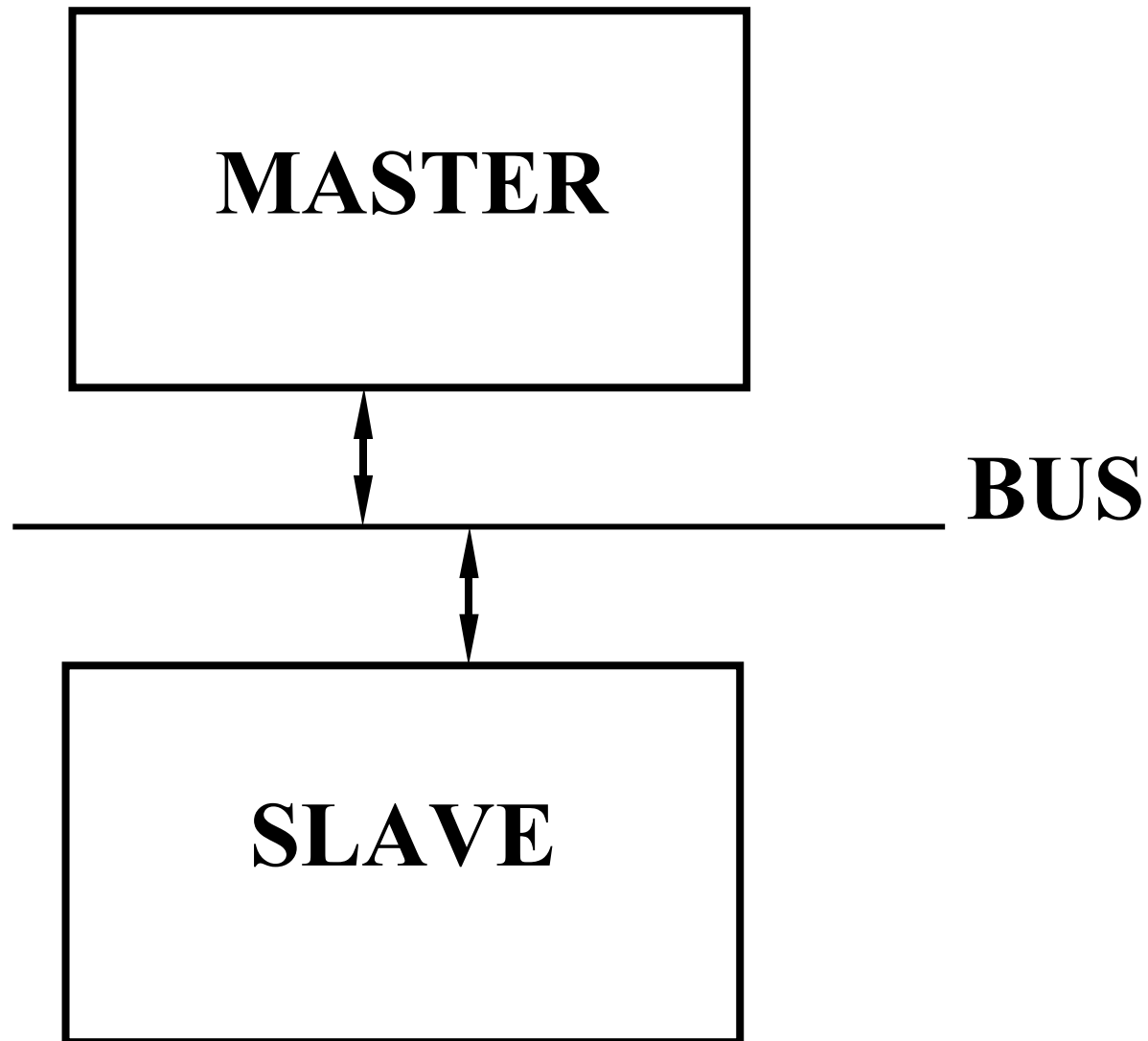
Viktiga delar i en dator



Rubriker

- **Bussöverföring – läsning och skrivning**
- Adressering – memory-mapped, I/O-mapped
- Parallellport – uppbyggnad och funktion
- Handskakning
- Serieport – funktion, paritetskontroll
- Eko-program, kommunikationsprogram
- Timer

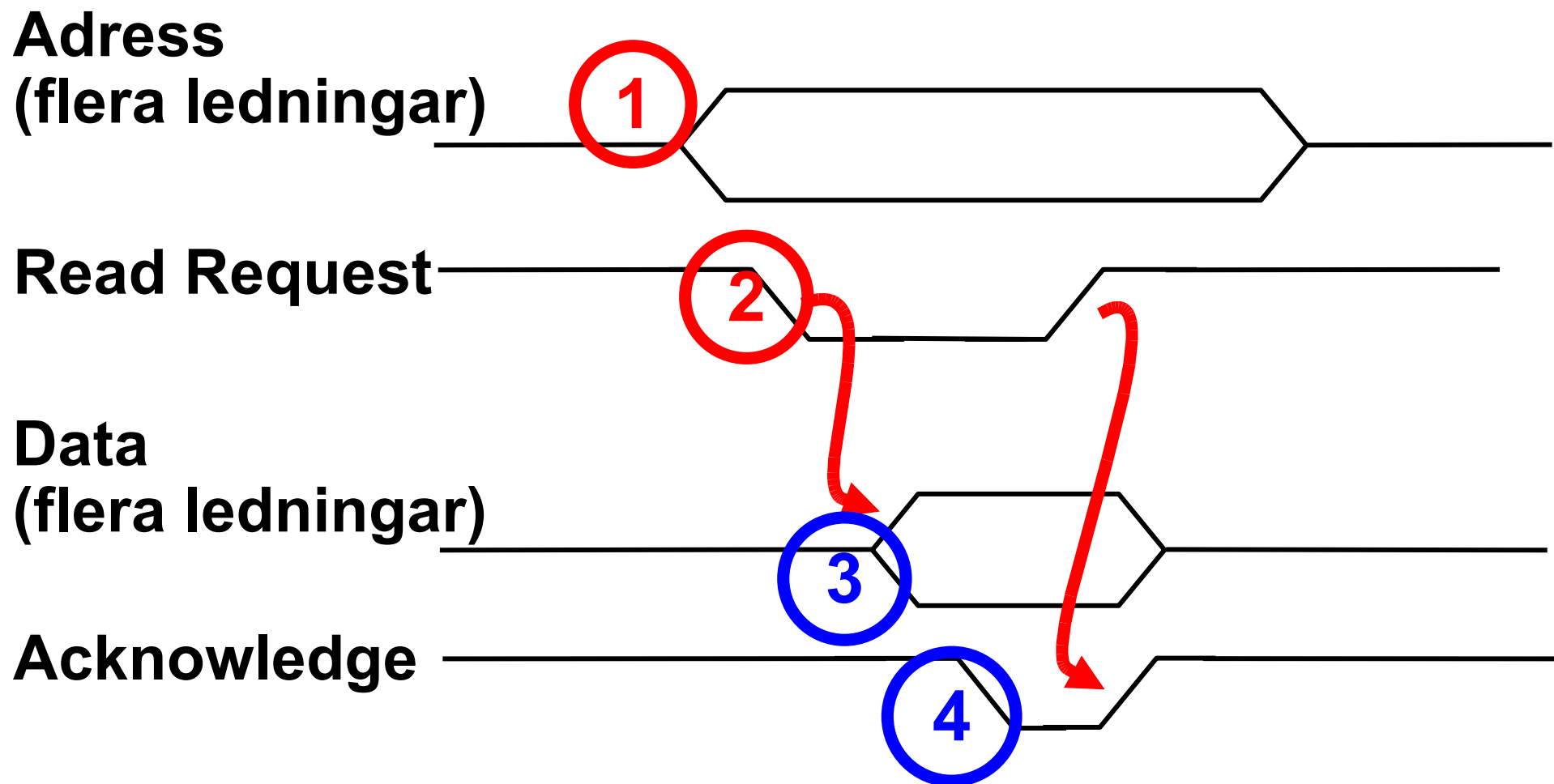
Bussöverföring



Exempel: memory read

- Master – till exempel processorn (CPU)
- Slave – till exempel minnet (MEM)
- Master levererar adress (Address)
- Master ger läsbegäran (Read Request)
- Slave levererar data
- Slave levererar kvittens (Acknowledge)

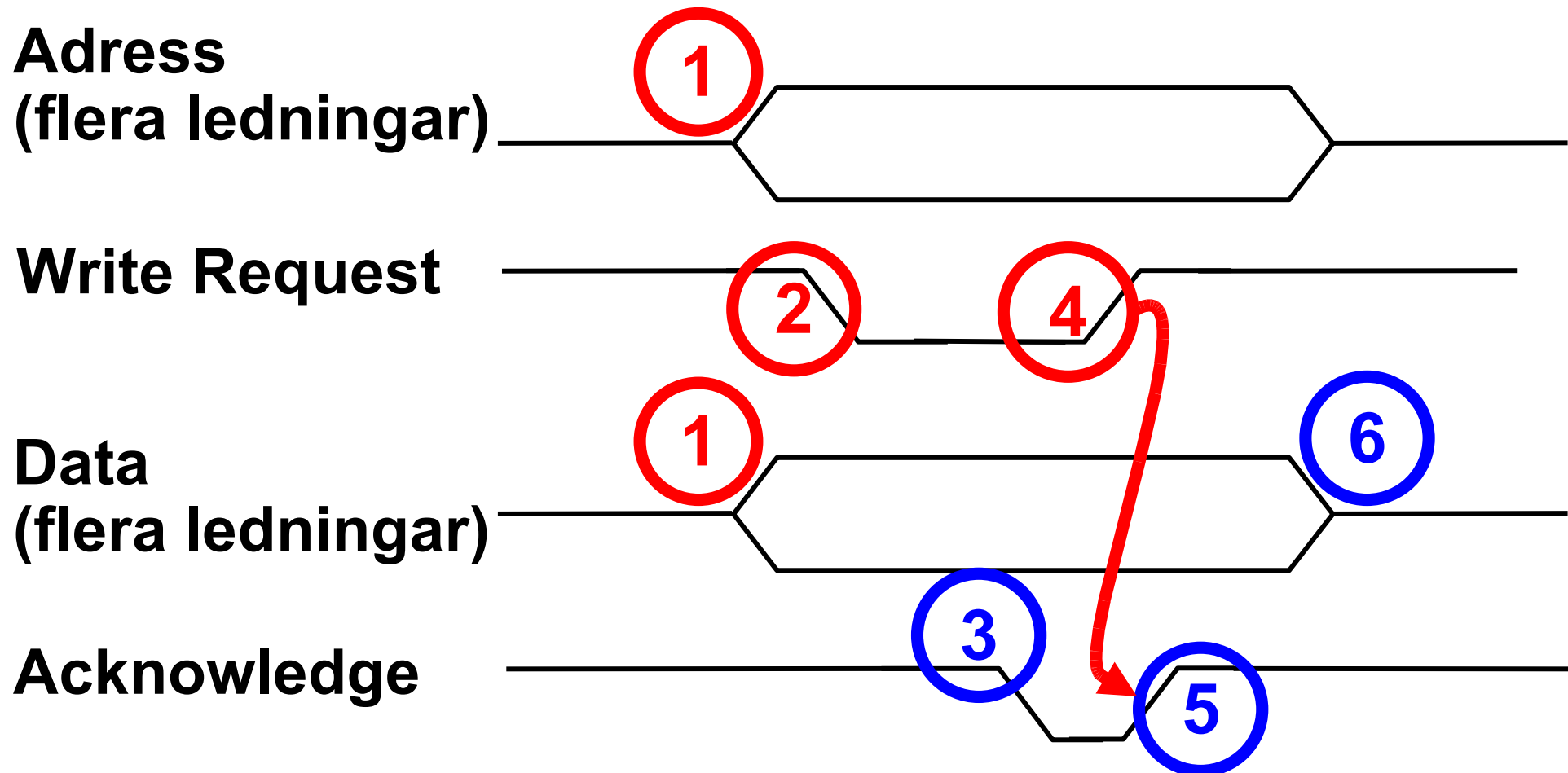
Exempel: memory read



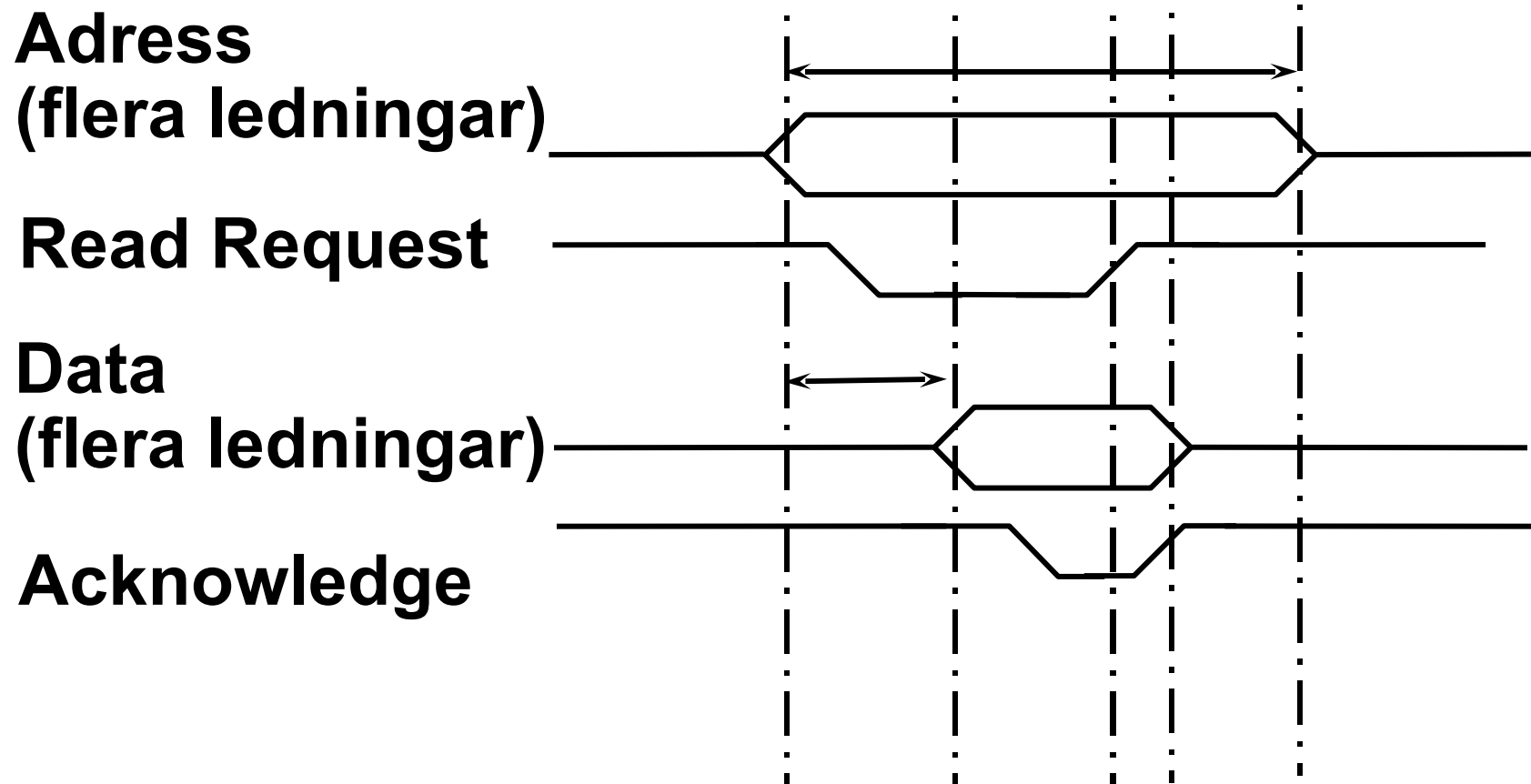
Exempel: memory write

- Master – till exempel processorn (CPU)
- Slave – till exempel minnet (MEM)
- Master levererar adress (Address) och Data
- Master ger skrivbegäran (Write Request)
- Slave levererar kvittens (Acknowledge)

Exempel: memory write

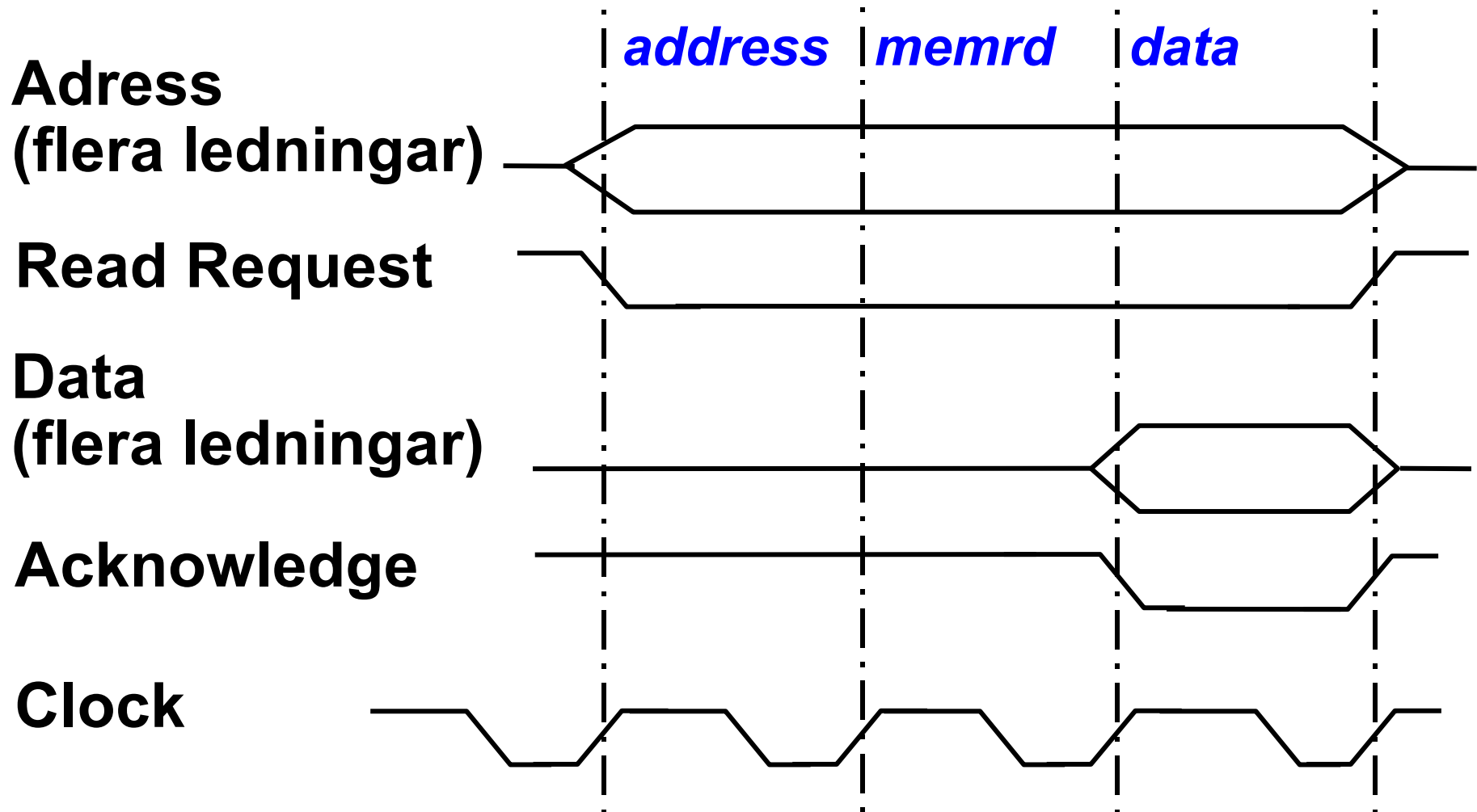


Asynkron buss



”any time”

Synkron buss



Asynkron eller synkron buss?

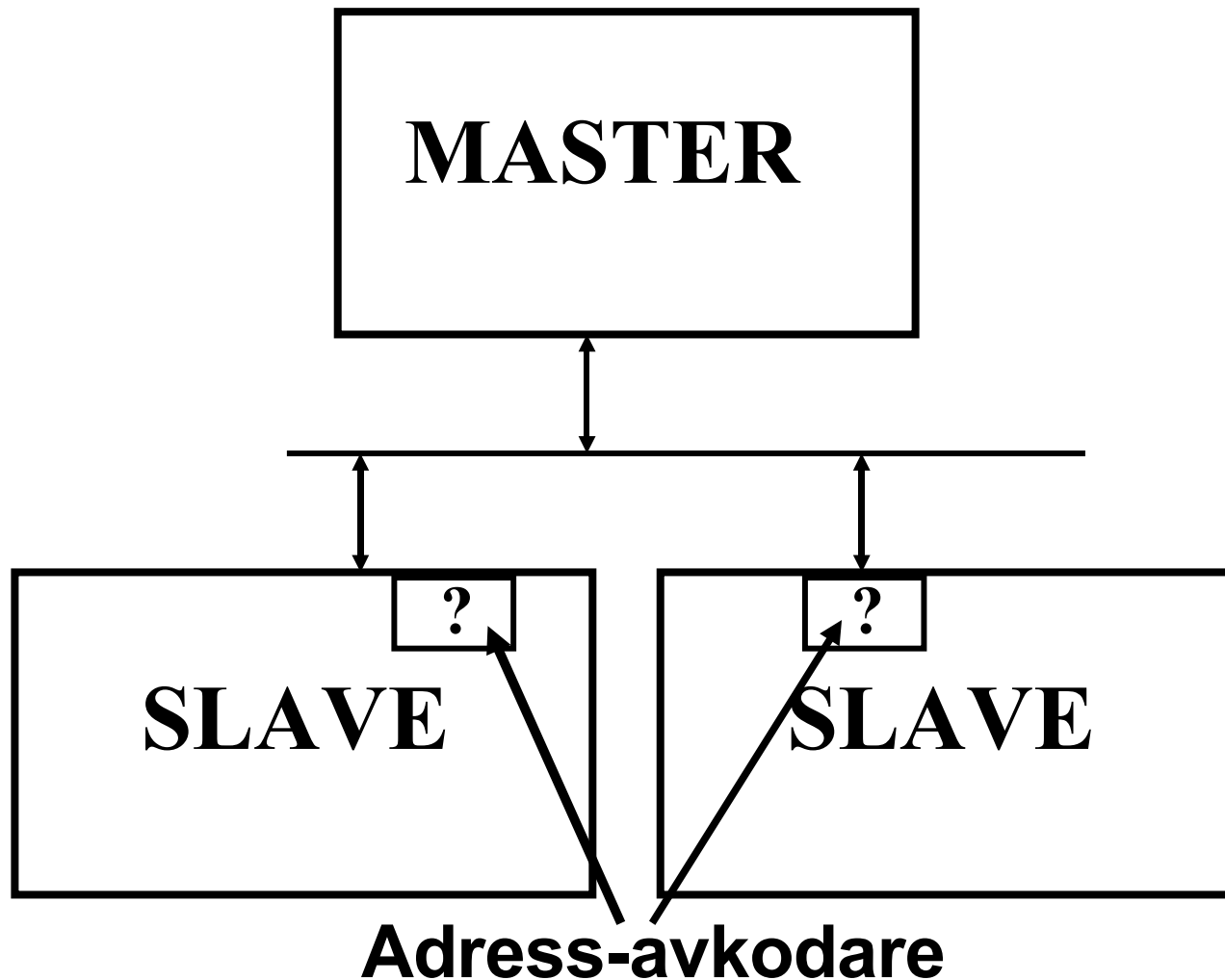
- Asynkron

- klarar enheter med olika hastighet
- bra för blandade och långsamma (äldre) system

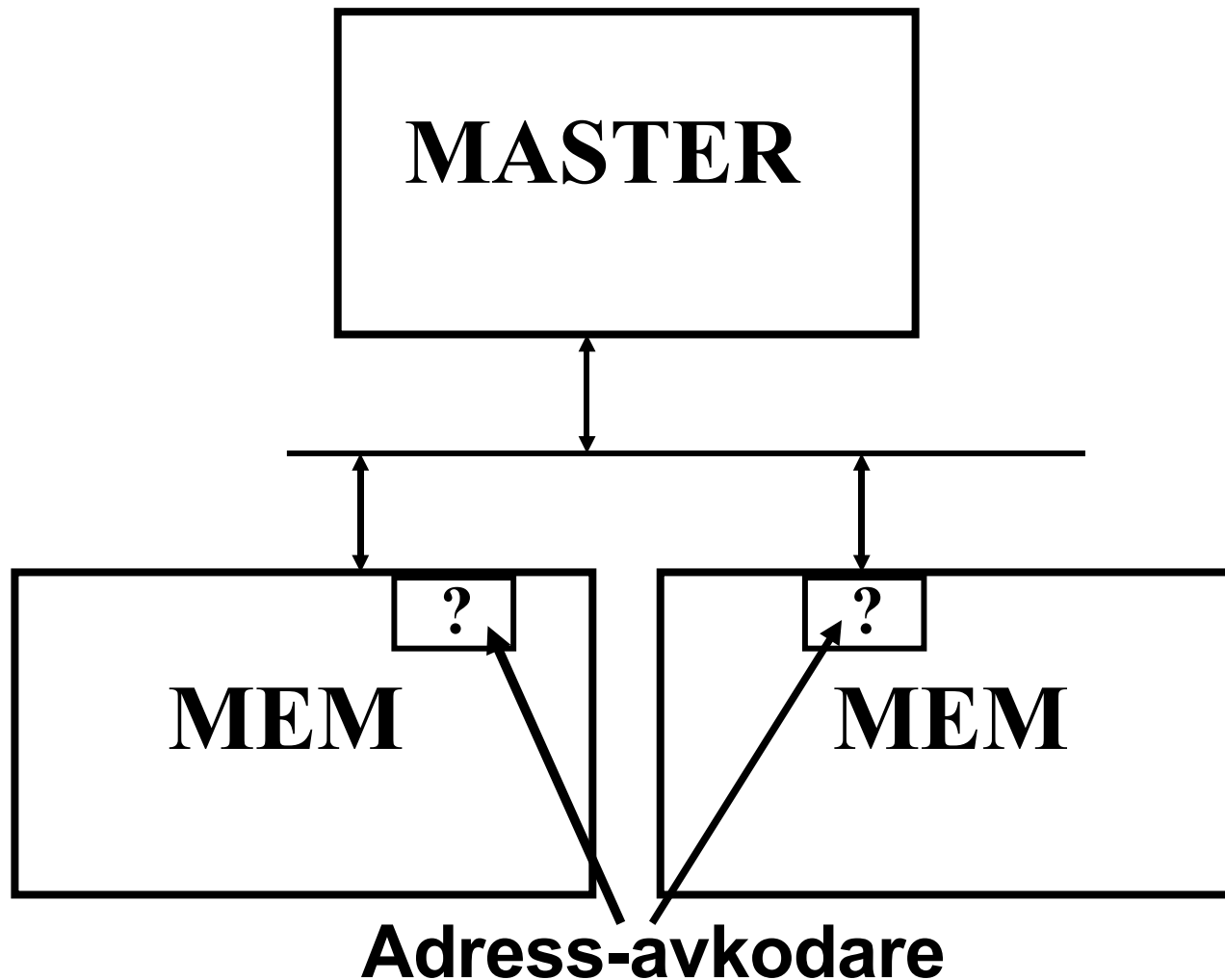
- Synkron

- kan ge högre hastighet
- kan överföra adress och data separat, tiden mellan kan utnyttjas till annat
- bra för snabba, likadana enheter

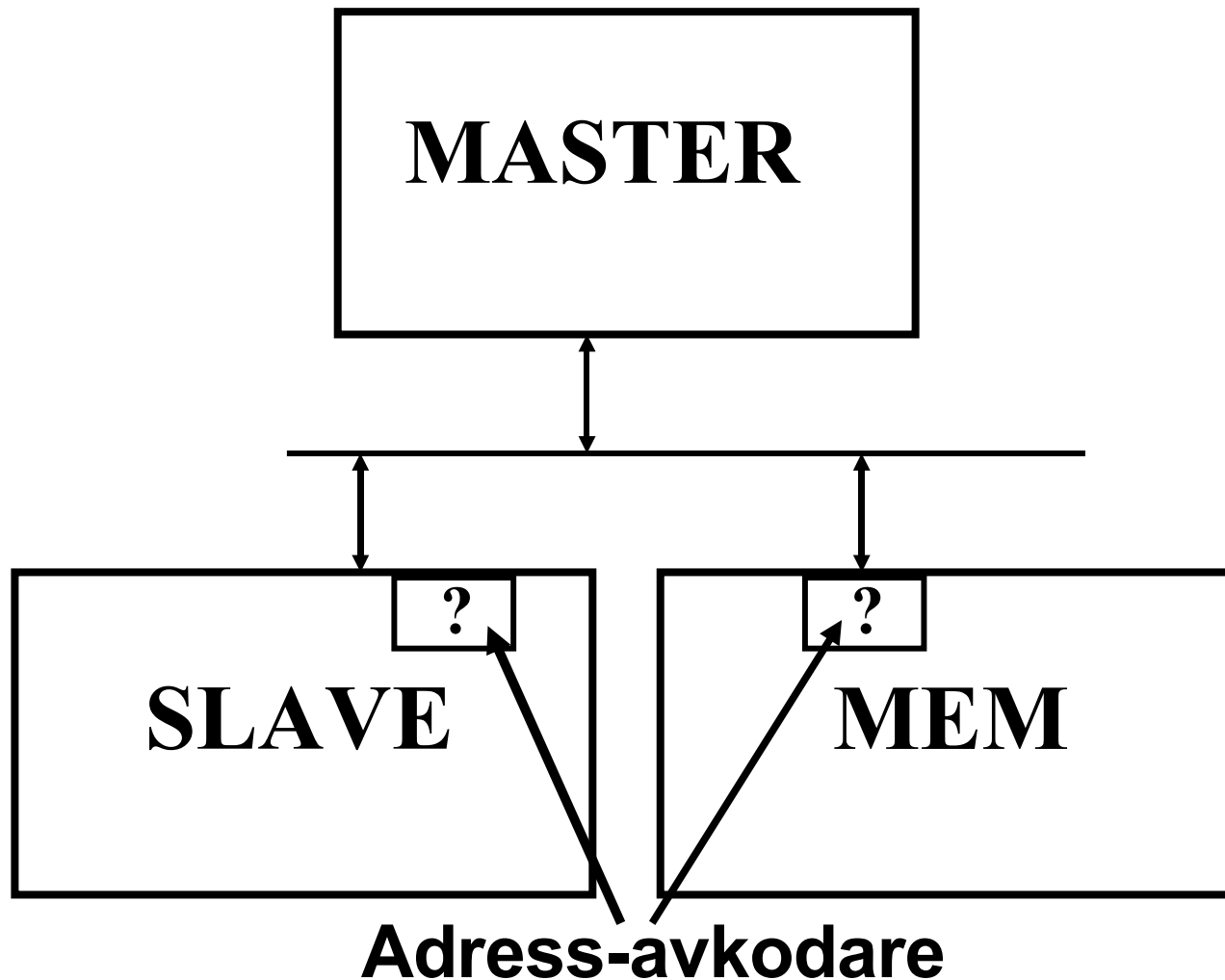
Flera *slave*-enheter inkopplade på olika adresser



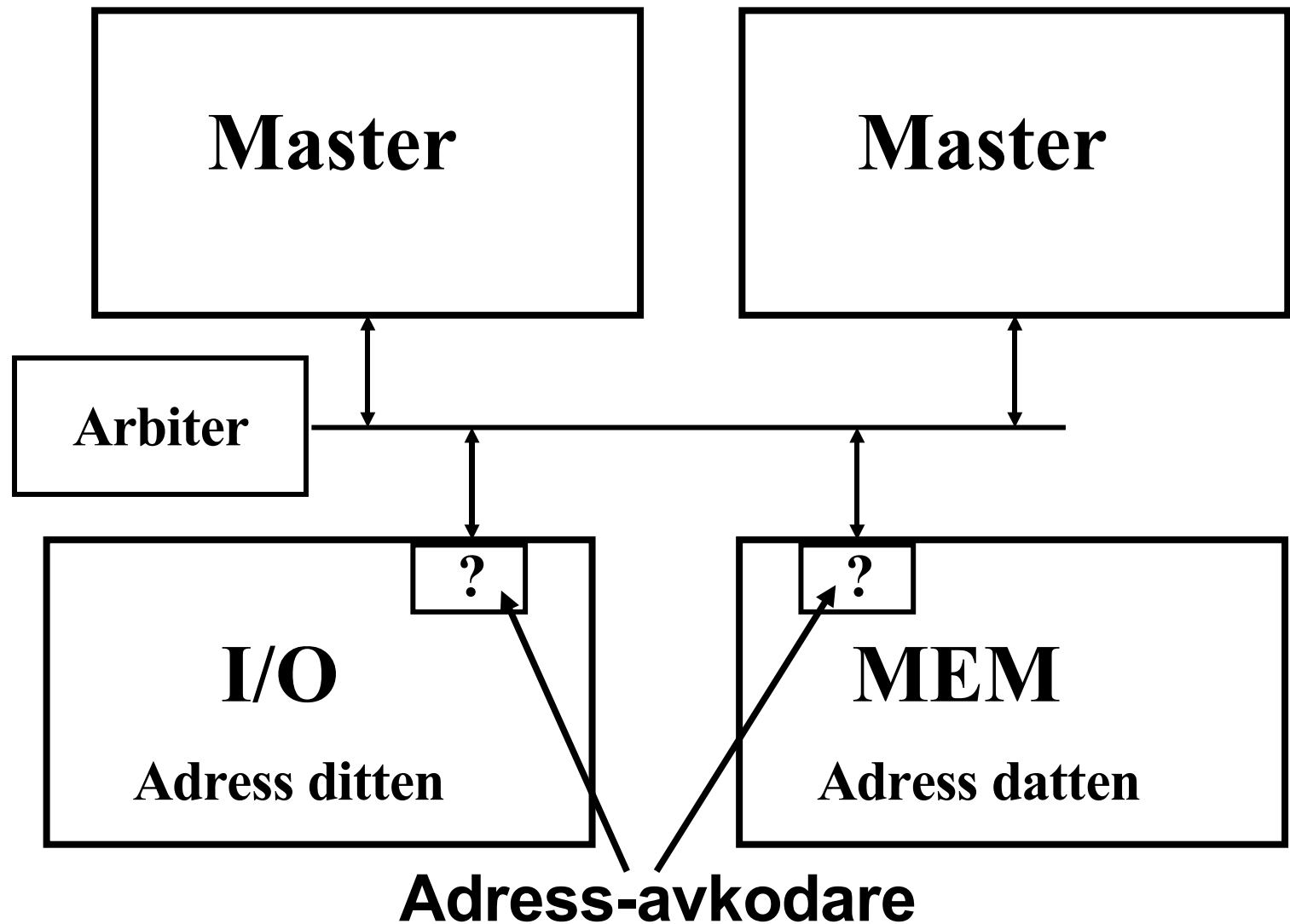
Flera *slave*-enheter inkopplade på olika adresser



Flera *slave*-enheter inkopplade på olika adresser



Flera Master går bra – Arbiter väljer mellan dem



Bussledningar

- Adressledningar: 8–64 stycken
 - för att kunna peka ut varje byte
- Dataledningar: 8–128 bitar
- Kontrolledningar för styrning
 - Read, Write (eller RD/WR och Strobe)
 - Acknowledge (eller motsatsen, Hold)
 - Bus Request, Bus Grant – för Arbiter
 - IRQ – Interrupt ReQuest
- Övrigt: Vcc, GND, Clock, Reset, ...

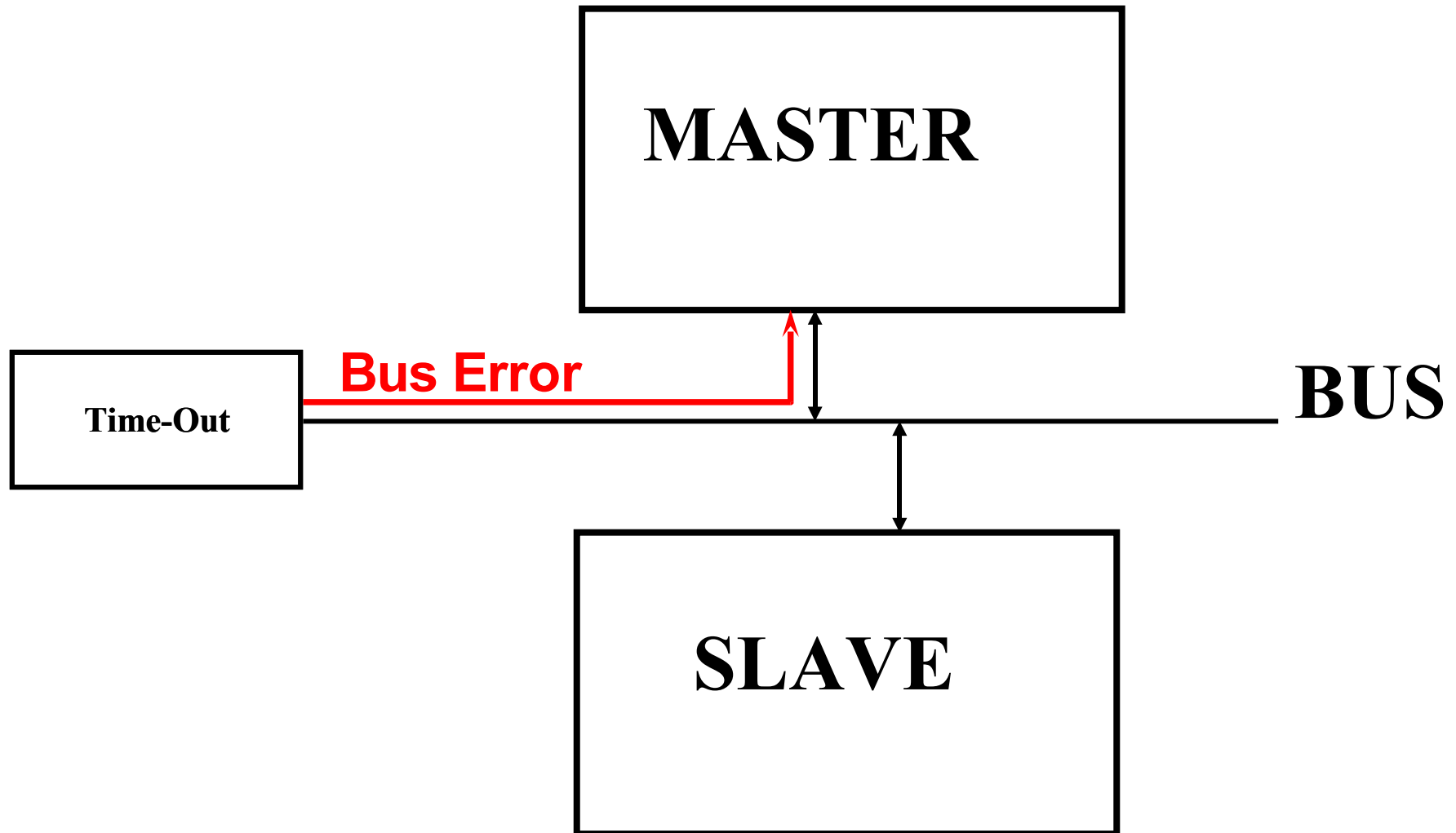
Bussledningar (forts.)

- Långsam Slave behöver extra tid vid läsning/skrivning
- Långsam Slave fördröjer ACK (eller aktiverar HOLD)
- Tidsövervakning, Time-Out, om Acknowledge aldrig kommer
- Bus Error – ingen ACK kom innan Time-Out

Bussledningar (forts.)

- Varje Slave måste ha adressavkodare med **olika** adresser så att bara en enhet svarar på varje request
- Läsning/skrivning av oanvänd adress medför Time-Out (och Bus Error) eftersom ingen enhet aktiverar Acknowledge

Bus Error: extra signal på bussen



Rubriker

- Bussöverföring – läsning och skrivning
- **Adressering – memory-mapped, I/O-mapped**
- Parallellport – uppbyggnad och funktion
- Handskakning
- Serieport – funktion, paritetskontroll
- Eko-program, kommunikationsprogram
- Timer

Inkoppling av in- och utenheter

- Antingen ***memory-mapped***...
 - in- och utenheter kopplas in på samma sätt som minne
 - LOAD och STORE för läsning och skrivning
- ...eller ***I/O-mapped***
 - in- och utenheter kopplas in med speciella, extra bussledningar
 - Nya instruktioner IN och OUT för läsning och skrivning

Inkoppling av in- och utenheter

- ***Memory-mapped***
 - Minne och I/O måste ha olika adresser
 - Minne och I/O har samma kontrollsignaler
 - Load, Store för I/O
- ***I/O-mapped***
 - Minne och I/O kan ha samma adresser
 - Minne och I/O har olika kontrollsignaler
 - Speciella in- och ut-instruktioner:
 - IN R1,117
 - OUT R2,118

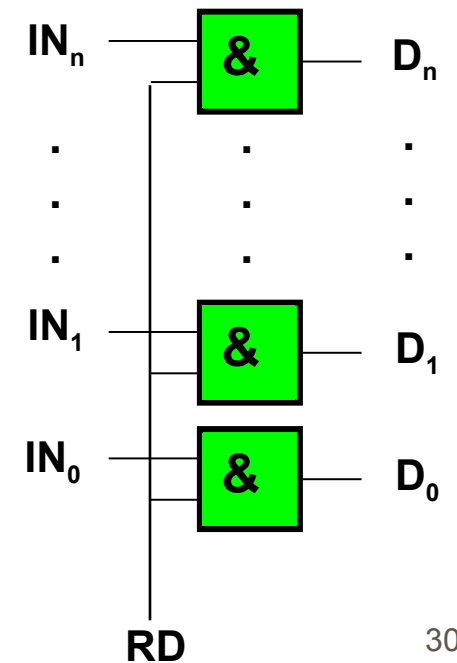
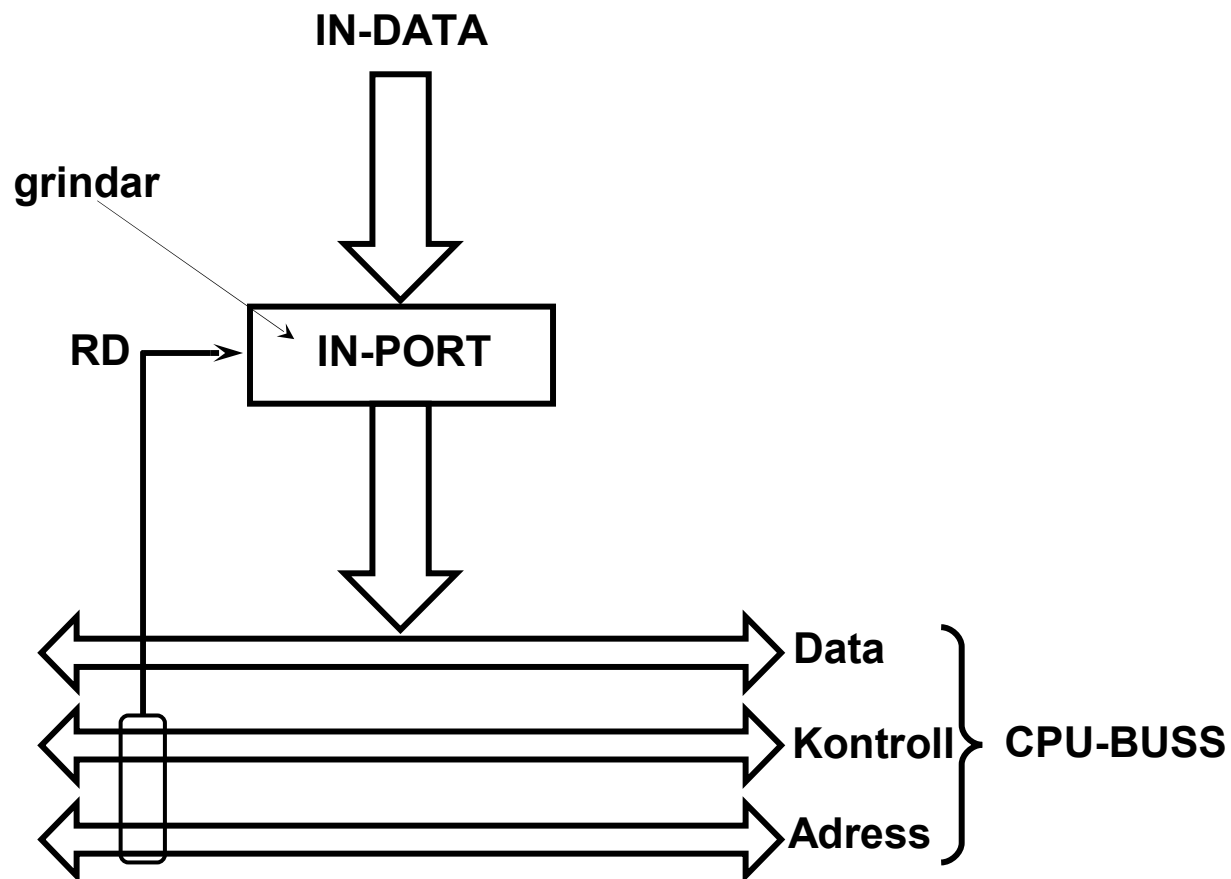
Rubriker

- Bussöverföring – läsning och skrivning
- Adressering – memory-mapped, I/O-mapped
- **Parallellport – uppbyggnad och funktion**
- **Handskakning**
- Serieport – funktion, paritetskontroll
- Eko-program, kommunikationsprogram
- Timer

Parallellport

- Brukar ha 8, 16, eller 32 bitar (anslutningar)
- Signalriktning: in eller ut
kan vara inställbar för varje bit
- Inport består normalt av grindar
- Utport består normalt av register
- Handskakningssignaler kan finnas

Parallell IN-port



IN-port

- Läsning öppnar grindarna
- Det värde som finns på inporten just då hämtas in till programmet
 - som resultat av LOAD- eller IN-instruktion
- Lämpligt för tryckknappar (till exempel)

IN-porten kopplad till annan dator

- IN-porten på min dator kan kopplas till UT-porten på en annan dator
- Filöverföring?

Exempel

- Texten "SOPPA" ska överföras
- Mottagaren ser värdet 01010000 extra länge
- Mottagaren kan inte veta om det är ett eller två "P"
- Resultat: "SOPA"

S

0 1 0 1 0 0 1 1

O

0 1 0 0 1 1 1 1

P

0 1 0 1 0 0 0 0

P

0 1 0 1 0 0 0 0

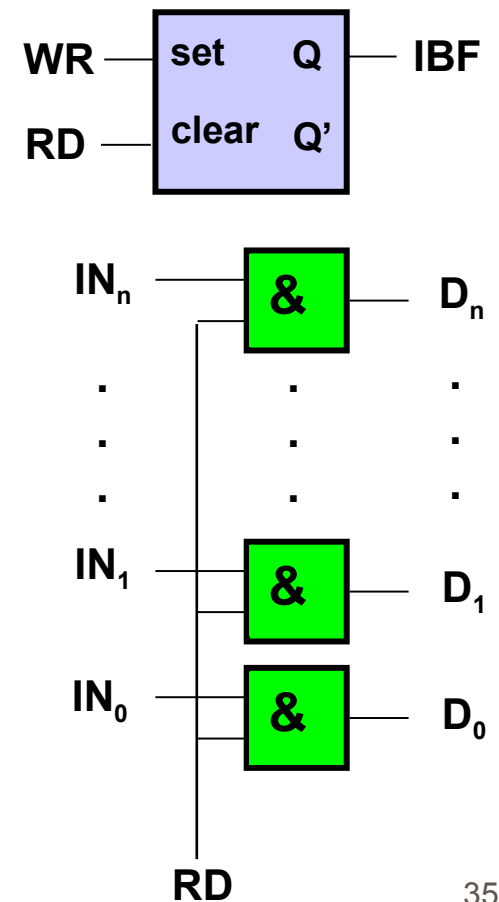
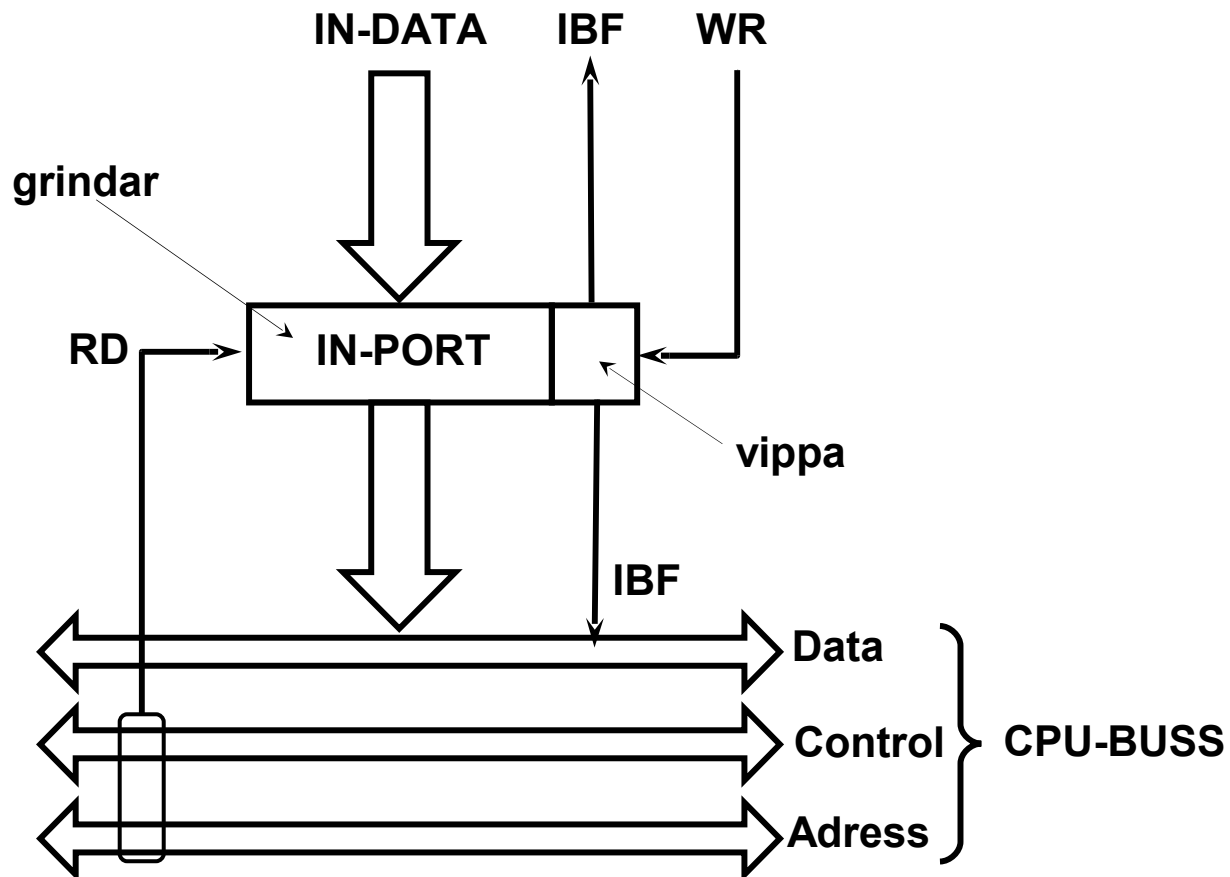
A

0 1 0 0 0 0 0 1

IN-port med handskakning

- Extra signaler, förutom data
- Till datorn: skrivsignal för inporten (WRITE)
- Från datorn: "Har ännu inte läst indata" = Input Buffer Full, IBF

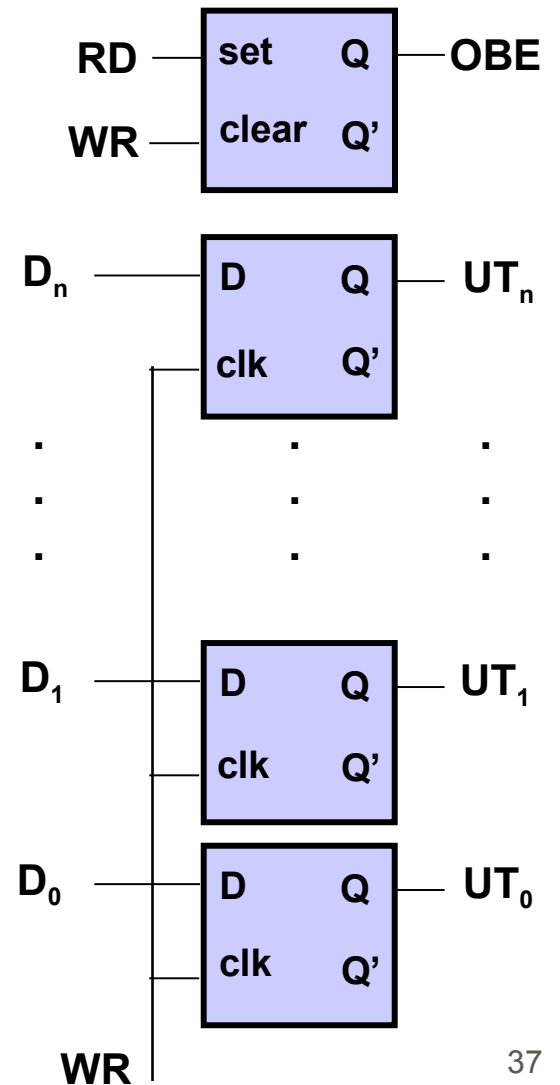
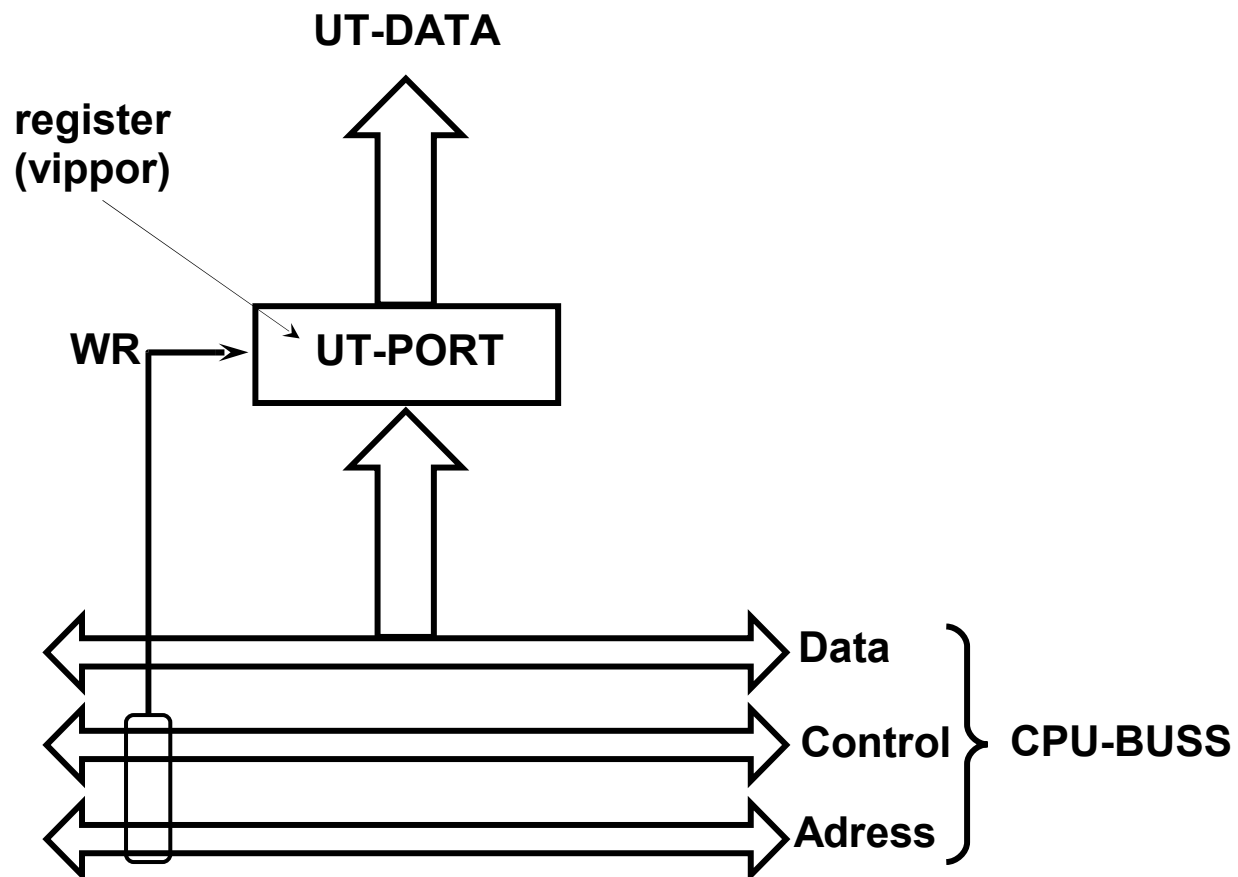
IN-port med handskakning



Handskakning

- IBF = Input Buffer Full
- Puls på WR ettställer IBF-vippan
- Utanför datorn finns något (någon?) som ger en puls på WR när det finns nya data
- När datorn läser inporten nollställs IBF-vippan av läs-signalen RD

Parallel UT-port



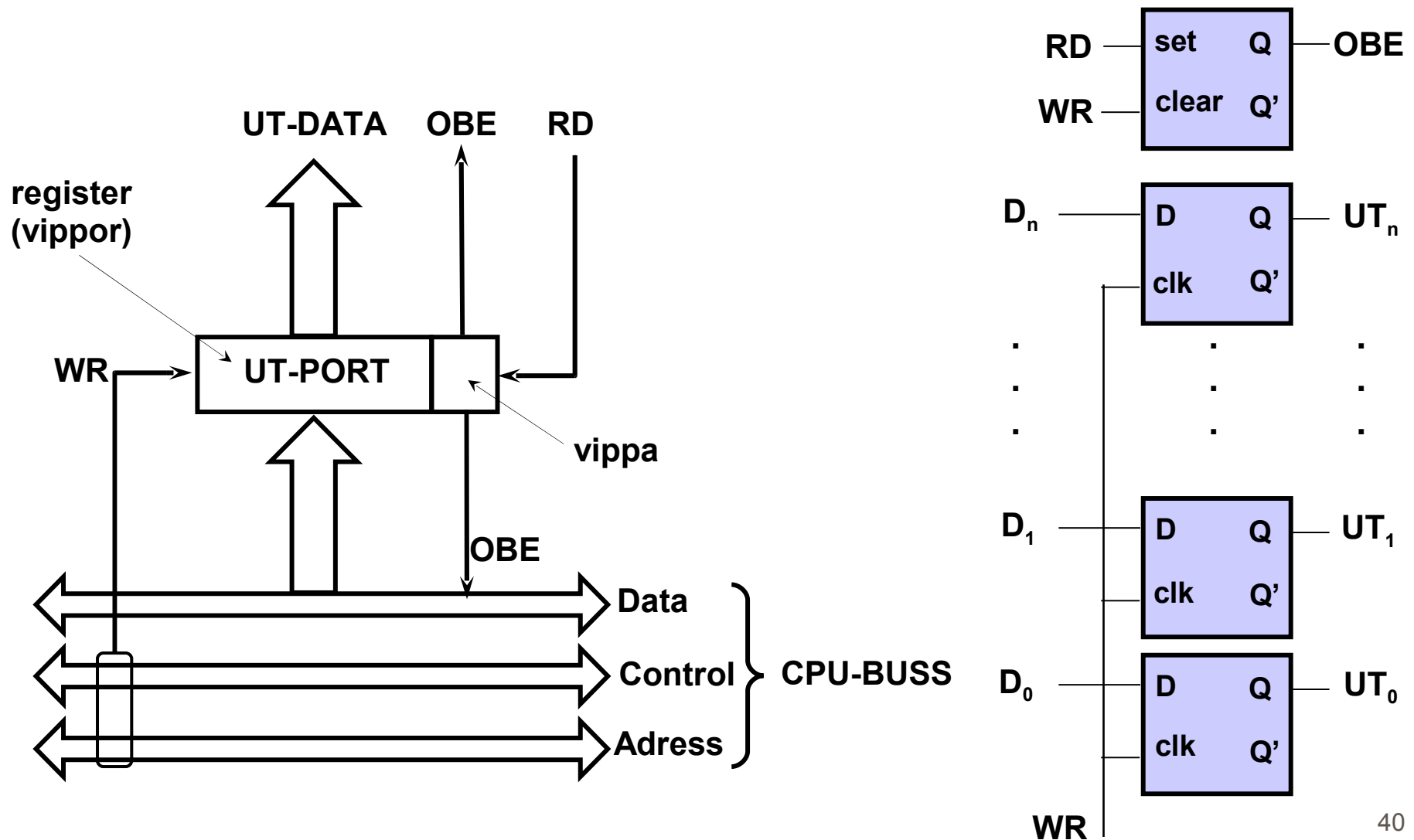
UT-port

- UT-porten innehåller vippor (till exempel 8, 16, 32 stycken) som tillsammans blir ett UT-register
- Datorn skriver till UT-registret
- Utanför datorn kan senaste värdet avläsas
- Lämpligt för lysdioder, motorer (till exempel)

UT-port med handskakning

- Extra signaler
- Till datorn: lässignal för utdata, READ
- Från datorn: "Finns ännu inga nya utdata" = Output Buffer Empty, OBE

UT-port med handskakning



Parallellport Handskakningssignaler

- IBF = Input Buffer Full
- OBF = Output Buffer Empty
- IBF och OBE är tillgängliga för program
- IBF och OBE är tillgängliga för hårdvaran

Handskakningssignaler (forts.)

- IBF = Input Buffer Full
- Aktiv signal IBF (full inport)
 - begäran om åtgärd, från omvärlden till program
 - "Hallå där, nu finns nya indata"
- Samma signal IBF (full inport)
 - varning till omvärlden, från datorn
 - "Stopp, datorn har ännu inte läst senaste indata"

Handskakningssignaler (forts.)

- OBE = Output Buffer Empty
- Aktiv signal OBE (tom utport)
 - begäran om åtgärd, från omvärlden till program
 - "Hallå där, leverera nya utdata"
- Samma signal OBE (tom utport)
 - varning till omvärlden, från datorn
 - "Stopp, datorn har ännu inte levererat nya utdata"

Handskakningssignaler: IBF

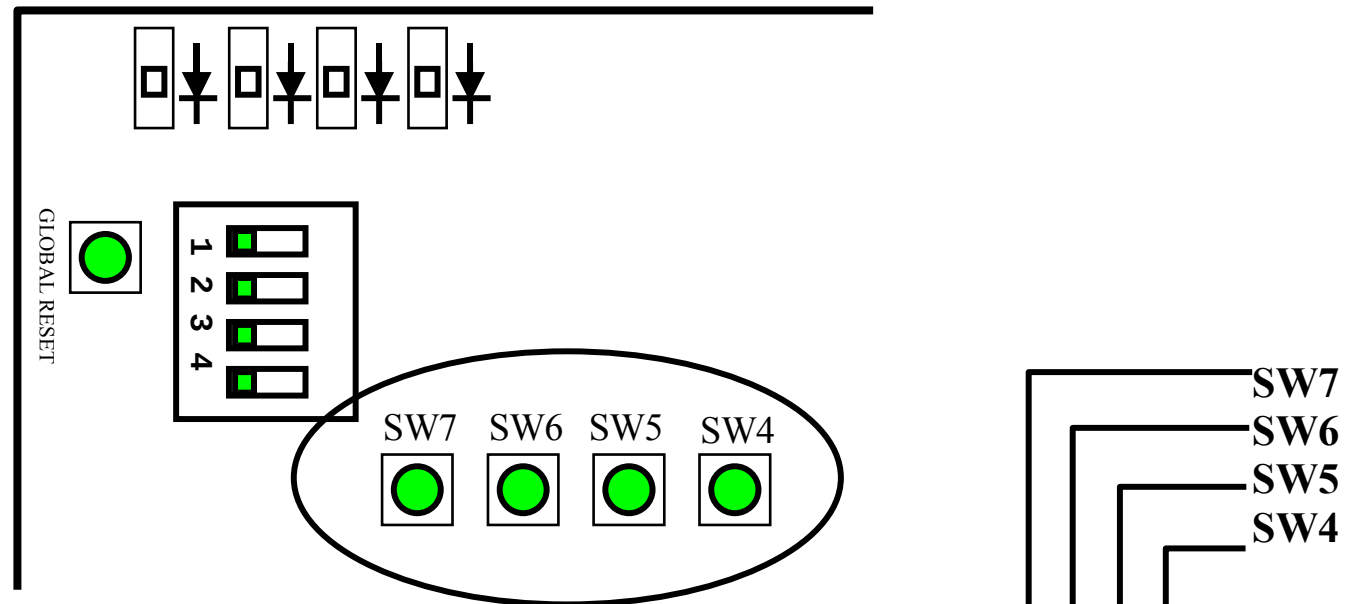
- IBF – Input Buffer Full
- IBF **ettställs** normalt av hårdvaran när omgivningen levererar nya data till inporten
- IBF **nollställs** (kvitteras) av program
- Två konstruktionstyper: nollställning kan ske
 - vid läsning av inport (implicit bieffekt), eller
 - med egen instruktion (explicit)

Handskakningssignaler: OBE

- OBE – Output Buffer Empty
- OBF **ettställs** normalt av hårdvaran när omgivningen har tagit hand om utdata
- OBE **nollställs** (kvitteras) av program
- Två konstruktionstyper: nollställning kan ske
 - vid skrivning av utport (implicit bieffekt), eller
 - med egen instruktion (explicit)

Nios2 button_pio Memory-mapped

Delar av
Nios2-kortet

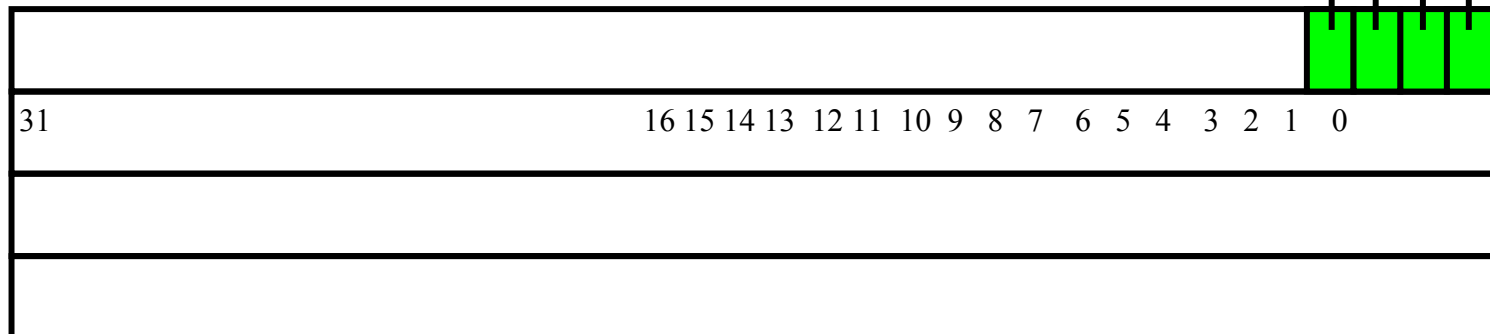


0x840

0x844

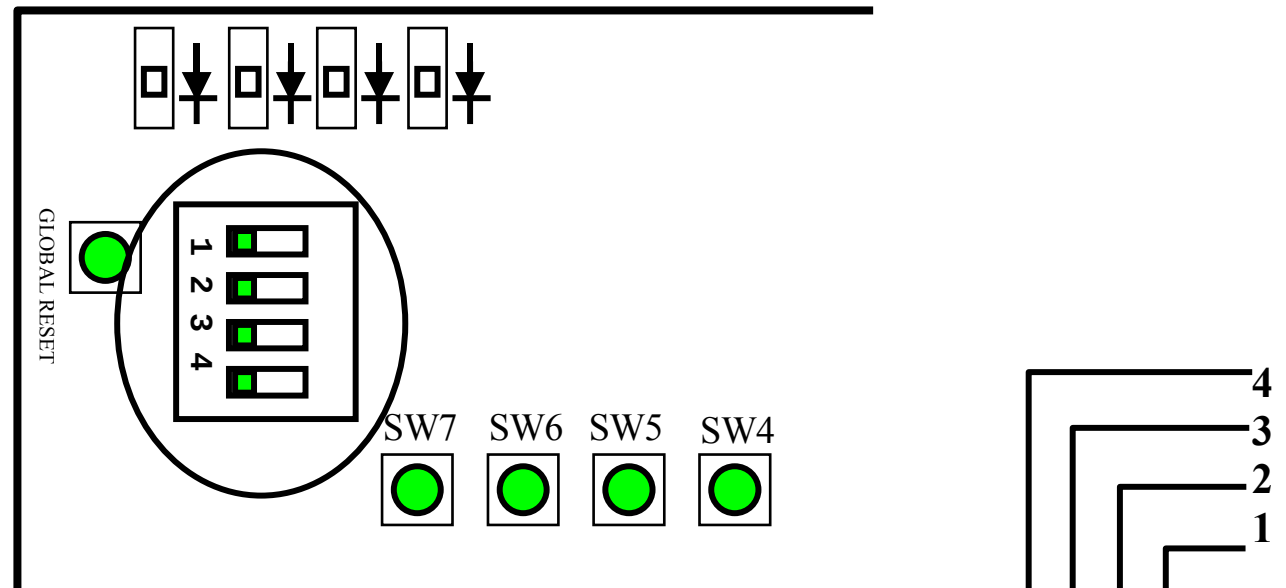
0x848

0x84C



Nios2 dip_pio Memory-mapped

Delar av
Nios2-kortet

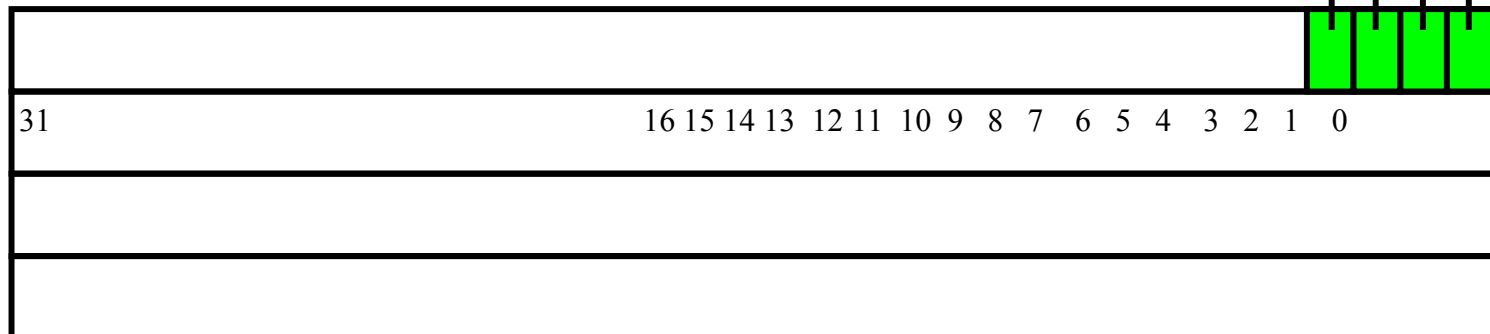


0x850

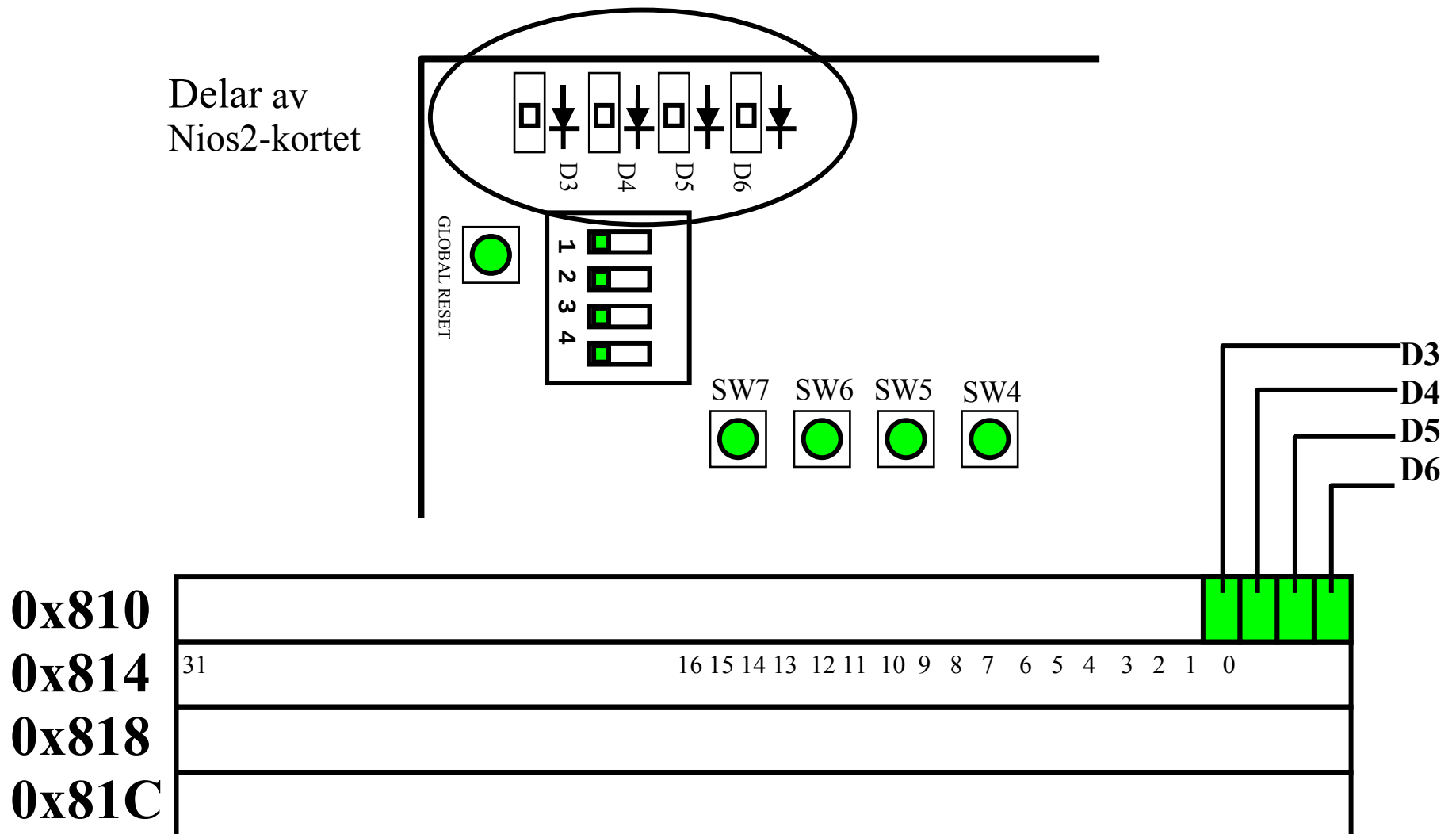
0x854

0x858

0x85C



Nios2 led_pio Memory-mapped



Minnesliknande beteende

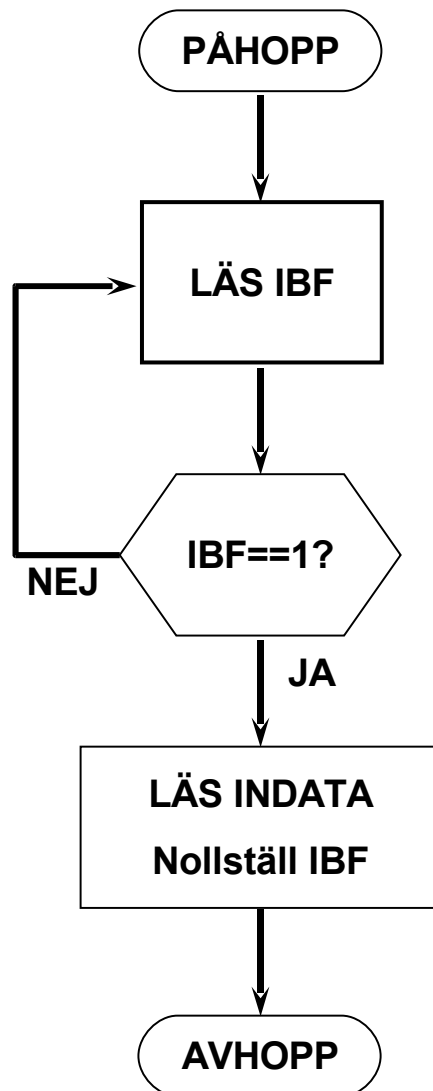
- **Så här ska minne fungera:**
- Ett värde som skrivs till en viss adress kan sedan läsas från samma adress
- Skrivning till en adress påverkar inte innehållet på andra adresser
- Läsning från en adress påverkar inte innehållet på någon adress

Icke minnesliknande beteende

- **Så här kan in- och utenheter fungera:**
- Ett värde som skrivs till en viss adress går inte att få tillbaka vid läsning från samma adress
- Skrivning till en adress kan påverka innehållet på flera andra adresser
- Läsning från en adress kan påverka innehållet på en eller flera adresser

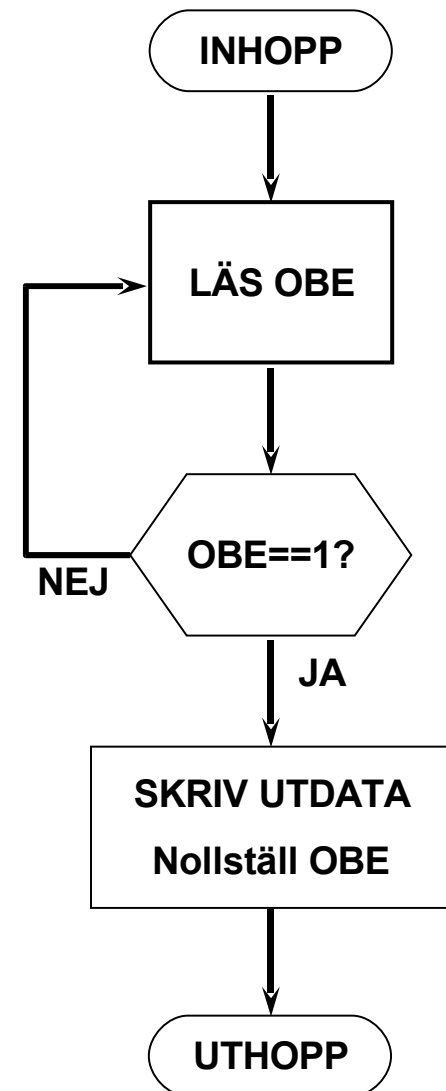
Program för handskakning

Invänta READY (blockerande I/O)



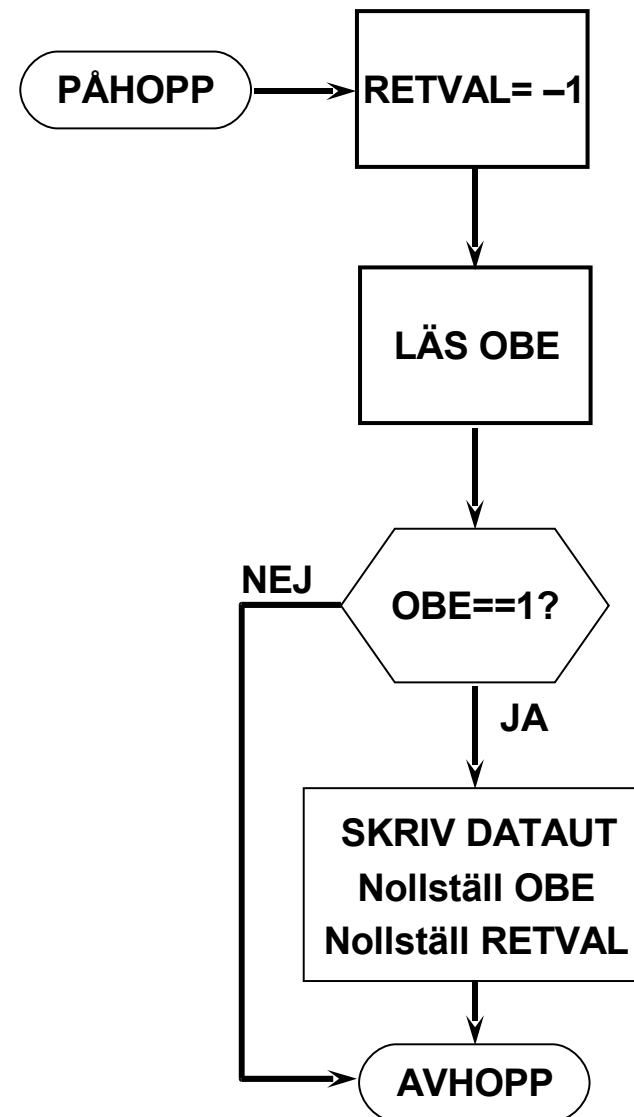
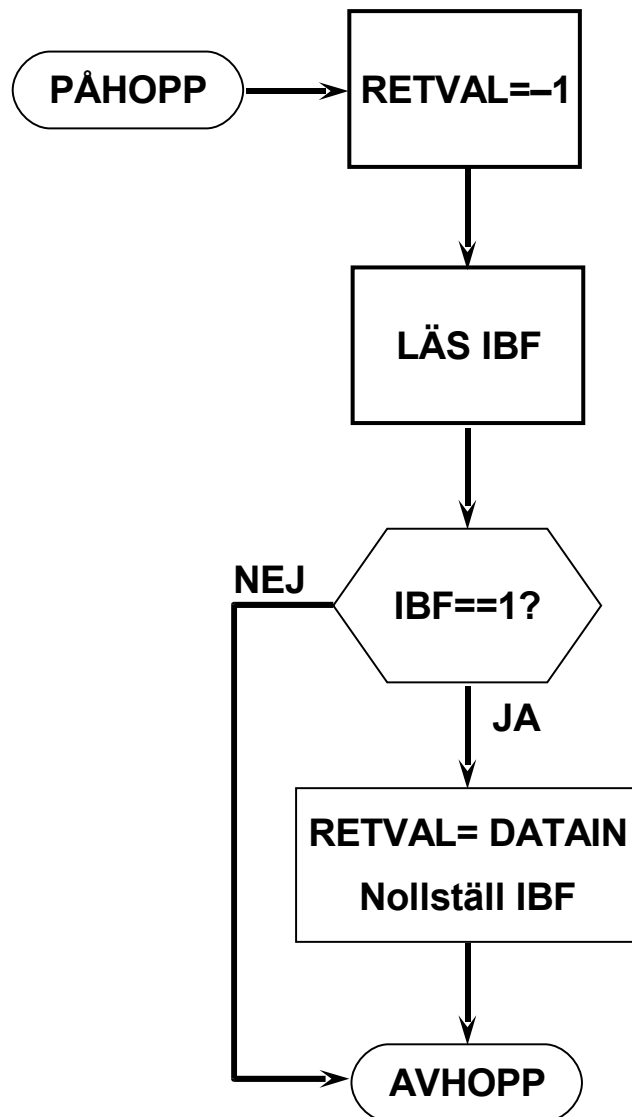
Wait for
READY

Action
&
Acknowledge !



Program för handskakning

Direkt retur (icke-blockerande I/O)



getdata från Inport rutin med pollning av statusbit (blockerande I/O)

```
# int in_data(reg)
define  datain  0x...
define  status  0x...
define  ibfmask 0x...

in_data:load    reg1 <-- status
              andi  reg1 <-- ibfmask
              beq   in_data
              load  reg <-- datain
              return
```

putdata till Utport

rutin med pollning av statusbit

(blockerande I/O)

```
# put_data(data)
define dataut 0x...
define status 0x...
define obemask 0x...

put_data: load  reg1 <-- status
           andi  reg1 <-- obemask
           beq   put_data
           store dataut <-- data
           return
```

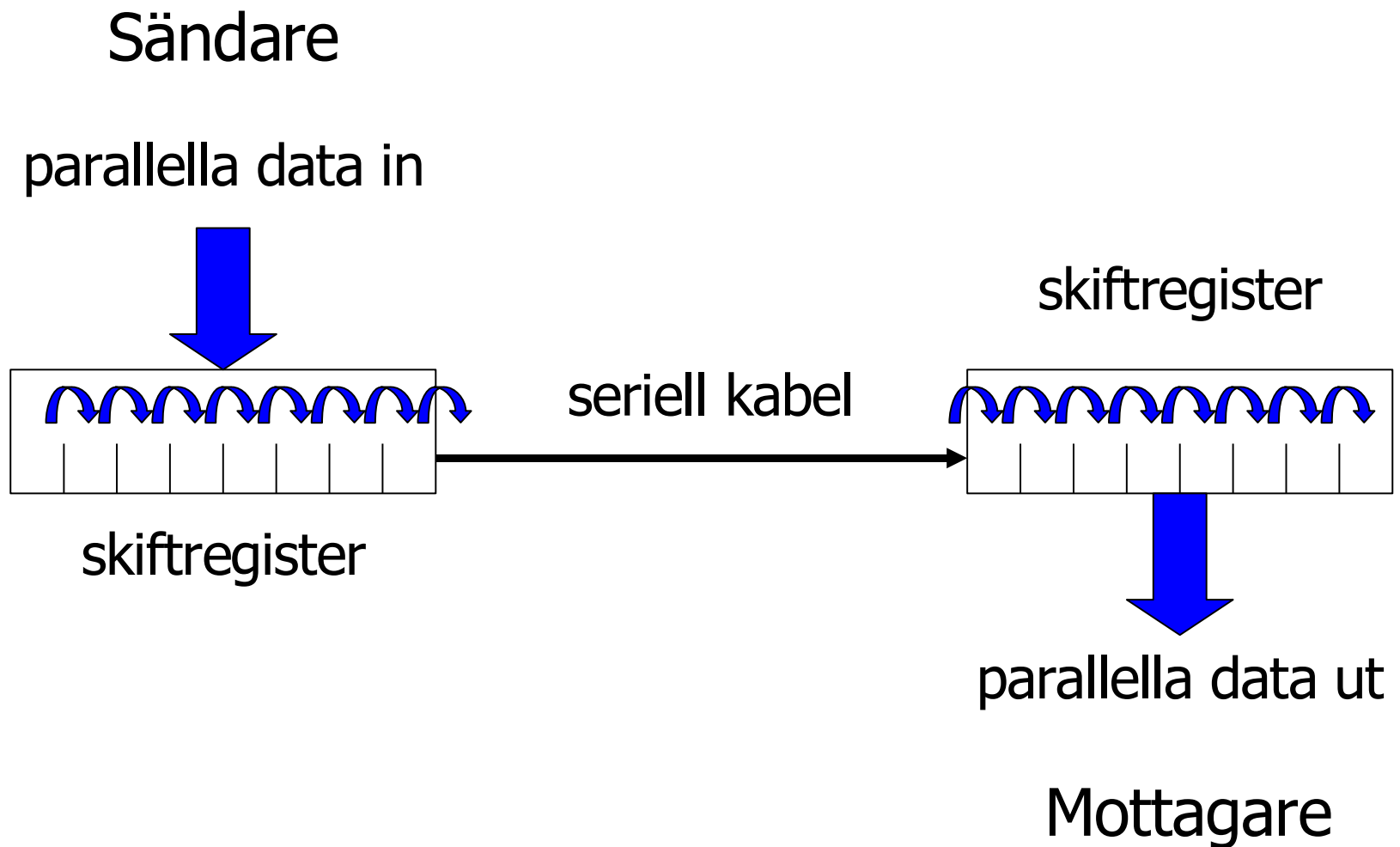
Rubriker

- Bussöverföring – läsning och skrivning
- Adressering – memory-mapped, I/O-mapped
- Parallellport – uppbyggnad och funktion
- Handskakning
- **Serieport – funktion, paritetskontroll**
- Eko-program, kommunikationsprogram
- Timer

Serieport

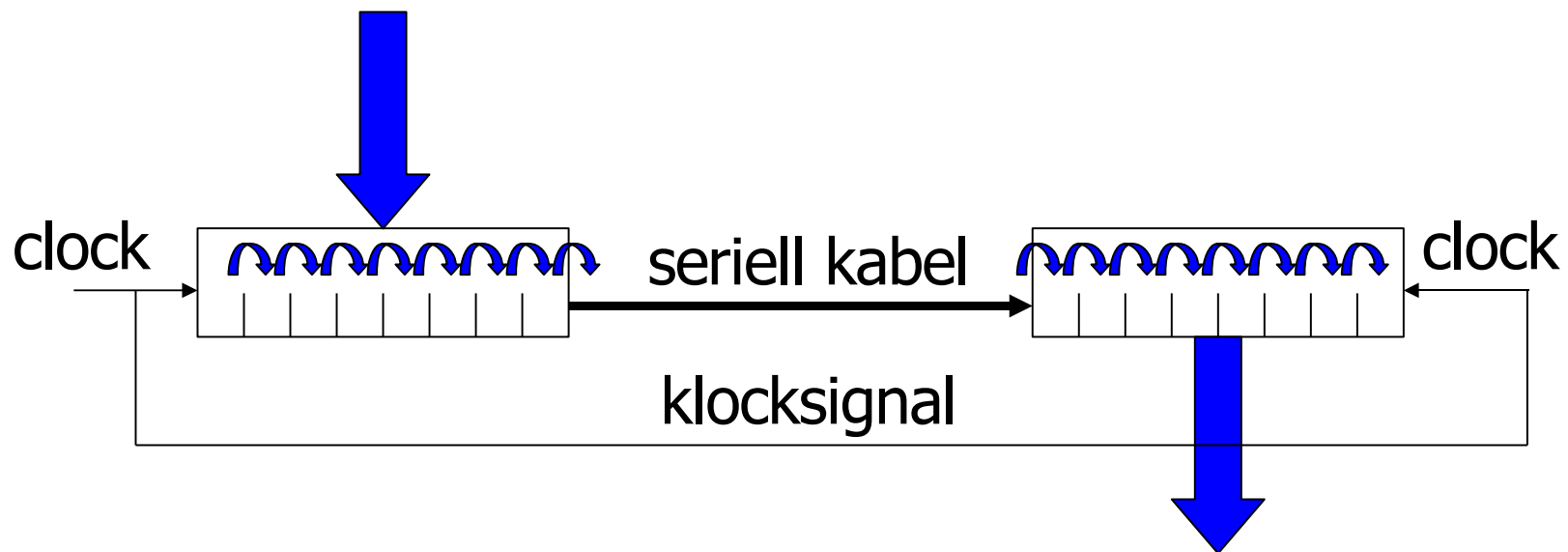
- Bit för bit i stället för flera bitar samtidigt
- Asynkron kommunikation vanligt
 - en byte kan komma när som helst
 - överföringslinjen är i vila mellan två data-bytes
- Synkron kommunikation – mera speciellt
 - det kommer alltid en ström av bitar
 - utfyllnadskoder sänds mellan två data-bytes

Seriell kommunikation



Synkron seriell kommunikation

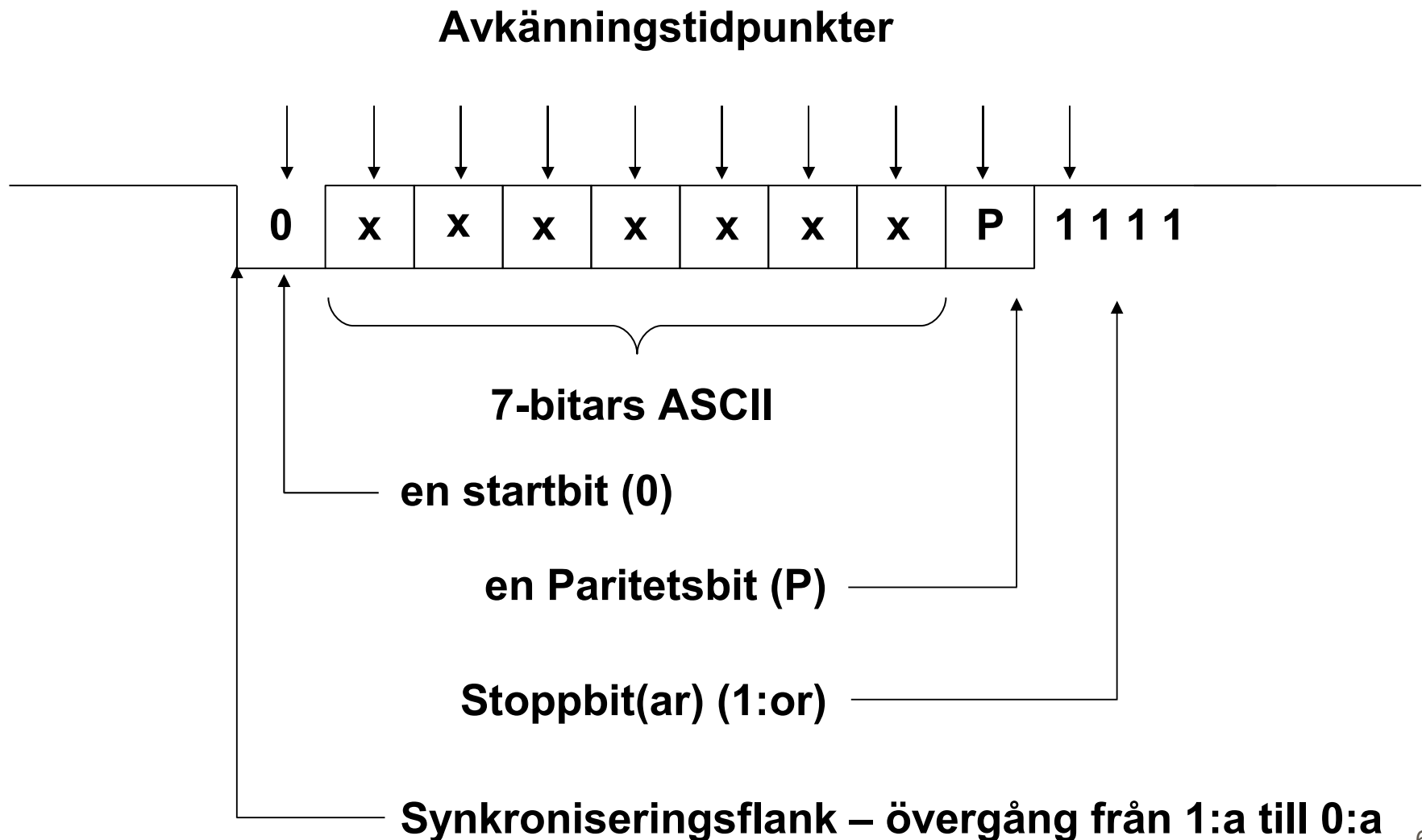
- Klocksignalen överförs på en extrasladd
- Synkroniseringskoder skickas då och då så att skiftregistren börjar på samma bit



Asynkron seriell kommunikation

- Varje byte (tecken, bokstav) sänds för sig
- Ett tecken kodas ofta med ISO-8859-1 men på labbarna använder vi 7-bits ASCII
- Dessutom tillkommer
 - 1 startbit (alltid 0:a)
 - 1 paritetsbit (tillval) – udda eller jämn paritet
 - 1, 1½ eller 2 stoppbitar (alltid 1:or)

Asynkron seriell kommunikation



Serieport med asynkron kommunikation

- Bithastighet – antal bitar per sekund
 - kallas felaktigt för baud-rate
- Teckenlängd – behöver inte vara 1 byte
 - kan vara 5, 6, 7 eller 8 bitar
- Paritetsbit (räknas inte in i teckenlängden)
 - av/på, jämn eller udda – enkel felkontroll
- Antal stoppbitar – minst en
 - flera stoppbitar ger längre vila mellan två bytes

Paritetskontroll

- Jämn paritet = jämnt antal ettor i ordet/byten
- Udda paritet = udda antal ettor i ordet/byten
- Fel detektering
 - upptäcker om 1 bit ändrat värde
- Sändaren kompletterar data med extra bit, värdet sätts så att pariteten blir rätt
- Mottagaren kontrollerar att pariteten är rätt

in_char från serieport

rutin med pollning av statusbit

```
# int in_char(void)
```

```
define data ...
```

```
define status ...
```

```
define ibfmask ...
```

```
in_char: load tmpreg <-- status
```

```
andi tmpreg <-- ibfmask
```

```
beq in_char
```

```
load outreg <-- data
```

```
return
```

putchar till serieport

rutin med pollning av statusbit

```
# out_char(data in inreg)
```

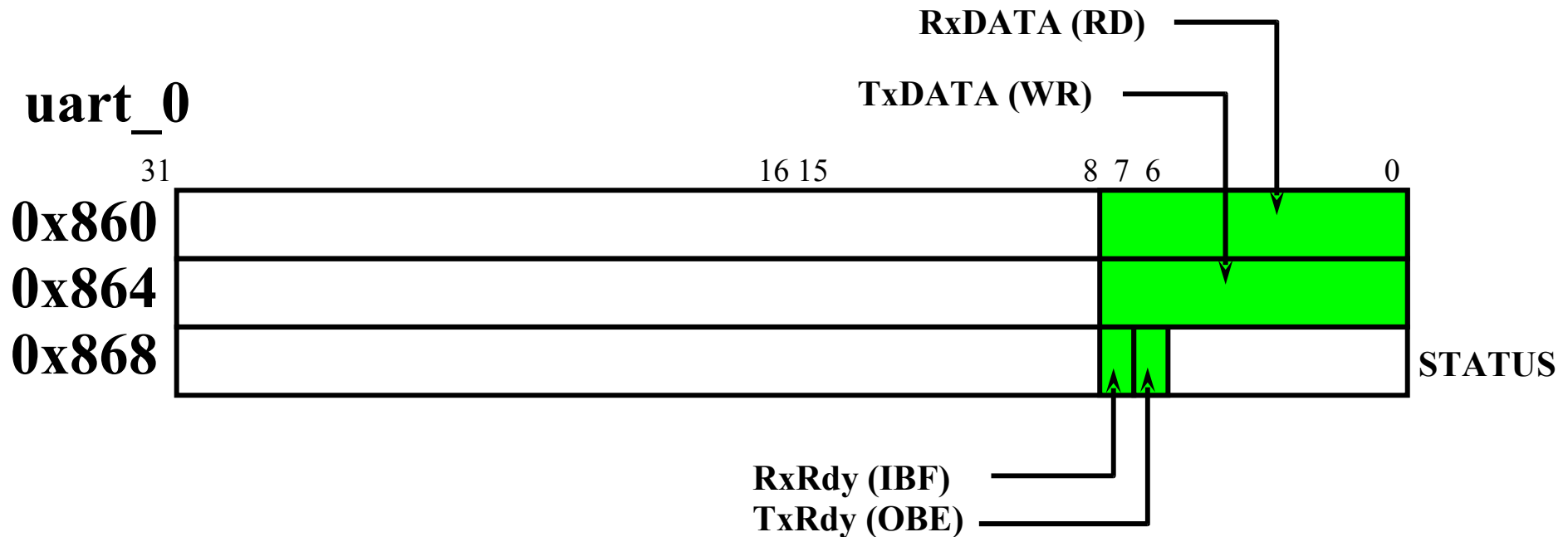
```
define dataout      ...
```

```
define status      ...
```

```
define obemask     ...
```

```
out_char: load      tmpreg <-- status  
           andi     tmpreg <-- obemask  
           beq      out_char  
           store    dataout <-- inreg  
           return
```


Nios2 uart_0 Memory-mapped



uart_0 **0x860-86F**
uart_1 **0x880-88F**

getchar från Serieport

syntetisk Nios-2-kod

(blocking)

```
# int returvärde = in_char(void)
```

```
.equ    datain,    0x860    # 0x880  
.equ    dataut,    0x864    # 0x884  
.equ    status,    0x868    # 0x888  
.equ    ibfmask,   0x80     # 0x80 = 0b10000000
```

```
in_char:  MOVIA      r8, datain  
          LDW        r9, 8(r8)      # status=datain+8  
          ANDI       r9, r9, ibfmask  
          BEQ        r9, r0, in_char  
          LDW        r2, 0(r8)  
          RET
```

out_char från Serieport syntetisk Nios-2-kod (blocking)

void out_char(parameter)

```
.equ    datain,    0x860      # 0x880  
.equ    dataut,    0x864      # 0x884  
.equ    status,    0x868      # 0x888  
.equ    obemask, 0x40        # 0x40=0b01000000
```

```
out_char: MOVIA      r8, datain  
          LDW        r9, 8(r8)      #status=datain+8  
          ANDI       r9, r9, obemask  
          BEQ        r9, r0, out_char  
          STW        r4, 4(r8)      # dataut=datain+4  
          RET
```

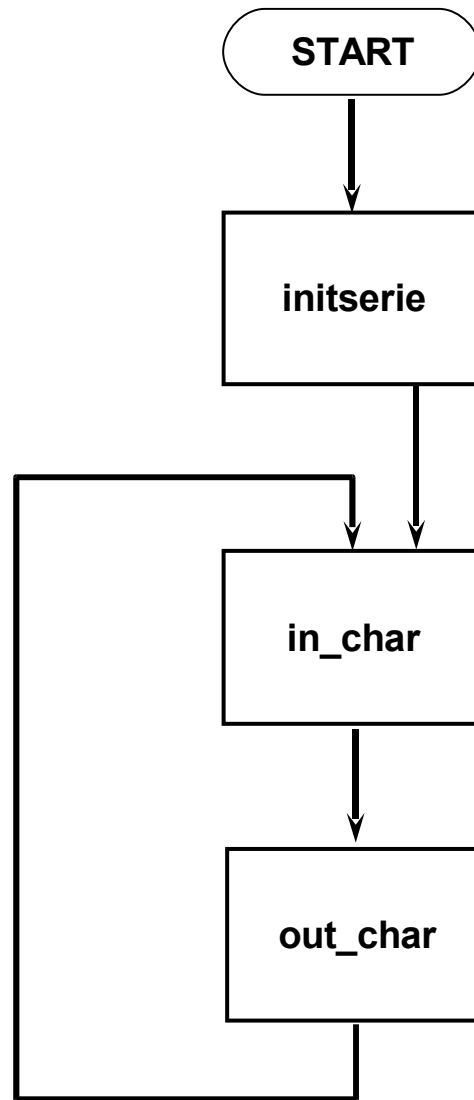
Rubriker

- Bussöverföring – läsning och skrivning
- Adressering – memory-mapped, I/O-mapped
- Parallellport – uppbyggnad och funktion
- Handskakning
- Serieport – funktion, paritetskontroll
- **Eko-program, kommunikationsprogram**
- Timer

Laboration 2: eko-program

- Skriv ett program som ekar tillbaka bytes från tangentbordet till hyperterminalfönstret
- main: call initserie # vid behov
- loop: call in_char # läs en byte
 mov r4,r2 # kopiera
 call out_char # sänd tillbaka
- br loop # loopa

Flödesschema för eko-program



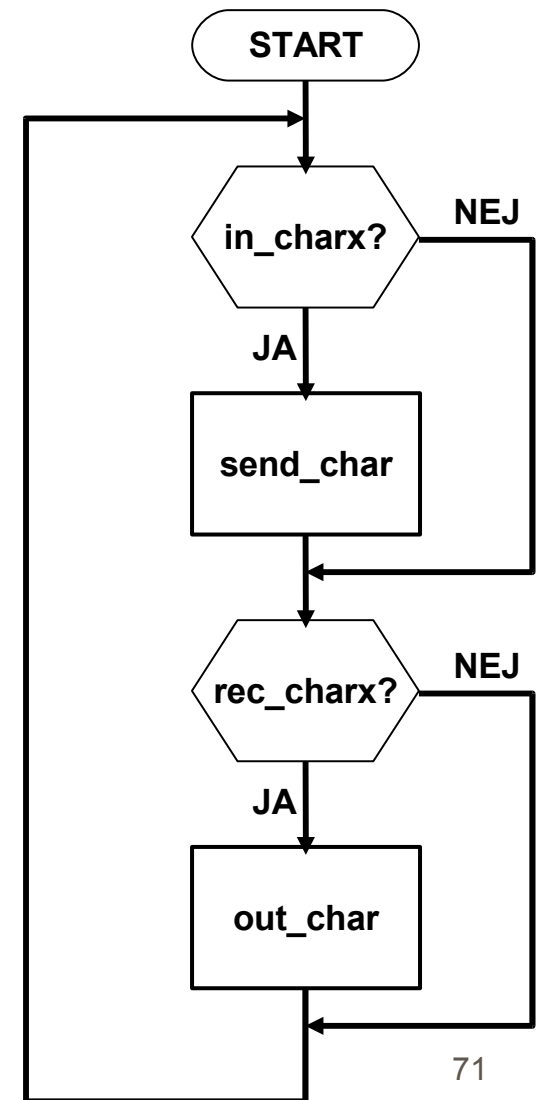
Flödesschema för korsvis kommunikation

nytt tecken från tangentbord?

om ja: sänd ut tecknet på länken

nytt tecken från länken?

om ja: skriv tecknet till skärmen

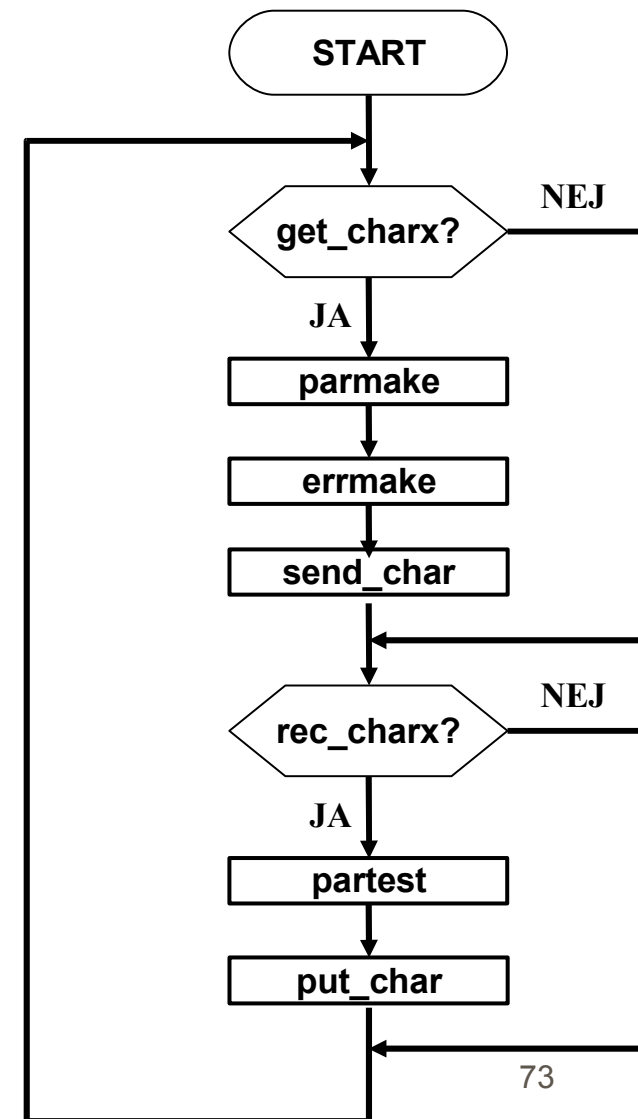


Program för korsvis kommunikation

- start: call initserie # initera serieportar vid behov
- gettst: call in_charx # tecken från tangentbord ?
 blt r2, r0, rectst # om inte: hopp till rectst
- send: mov r4, r2
- call send_char # sänd ett tecken
- rectst: call rec_charx # tecken mottaget ?
 blt r2, r0, gettst # om inte: hopp till gettst
- put: mov r4, r2
- call out_char # och skriv ut mottaget tecken
- br gettst # loopa

Korsvis kommunikation med paritetskontroll

- ***nytt tecken från tangentbord?***
 - ***om ja: beräkna checksumma
simulera slumpmässiga fel
sänd ut tecknet på länken***
- ***nytt tecken från länken?***
 - ***om ja: kolla checksumma
skriv tecknet till skärmen***



Korsvis kommunikation med paritetskontroll

- start: call initserie # initera serieportar vid behov
- gettst: call get_charx # tecken läst från tangentbord?
blt r2, r0, rectst # om inte: hopp till rectst
- mov r4, r2 # annars: beräkna checksumma
call parmake
mov r4, r2 # och simulera slumpmässiga fel
call errmake
mov r4, r2
- call send_char # sänd sedan ett tecken
- rectst: call rec_charx # tecken mottaget från kommunikationslänk?
blt r2, r0, gettst # om inte: hopp till gettst
- mov r4, r2 # annars: kontrollera checksumma
call partst
mov r4, r2
- call put_char # skriv slutligen ett tecken till skärmen
- br gettst # och loopa

Rubriker

- Bussöverföring – läsning och skrivning
- Adressering – memory-mapped, I/O-mapped
- Parallellport – uppbyggnad och funktion
- Handskakning
- Serieport – funktion, paritetskontroll
- Eko-program, kommunikationsprogram
- **Timer**

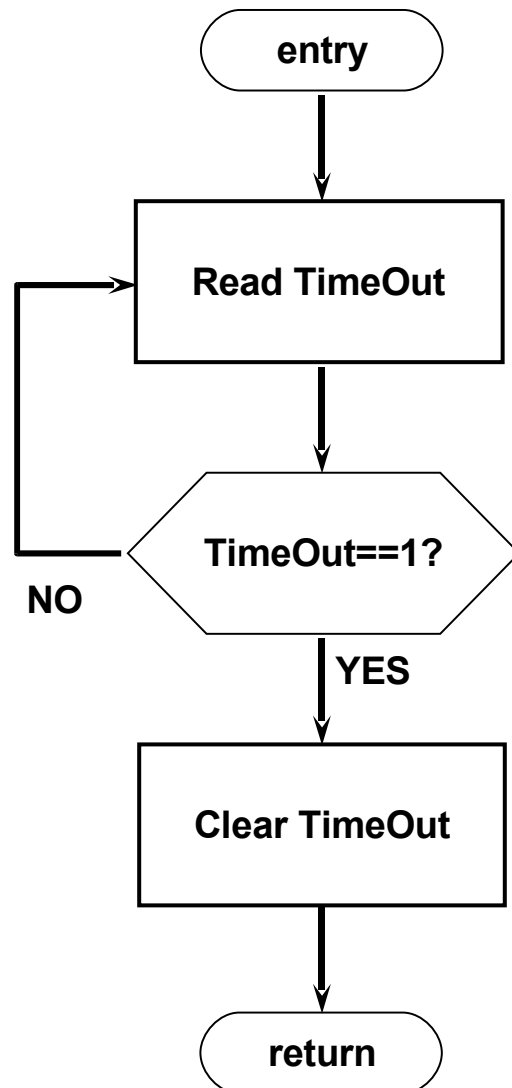
Timer

- Används för exaktare tidmätning
- Har en räknare (counter),
stegar 1 steg varje klockcykel
 - kan stega från 0 upp till inställt värde
 - eller stega från inställt värde ner till 0
- När räknaren har tickat klart: Time-Out
 - precis som en äggklocka

Timer: Time-Out

- Vid Time-Out ettställs en vippan
- Värdet i vippan är tillgängligt för program
- Värdet i vippan är tillgängligt elektriskt
- Innan nästa tidmätning ska vippan återställas
 - HUR?

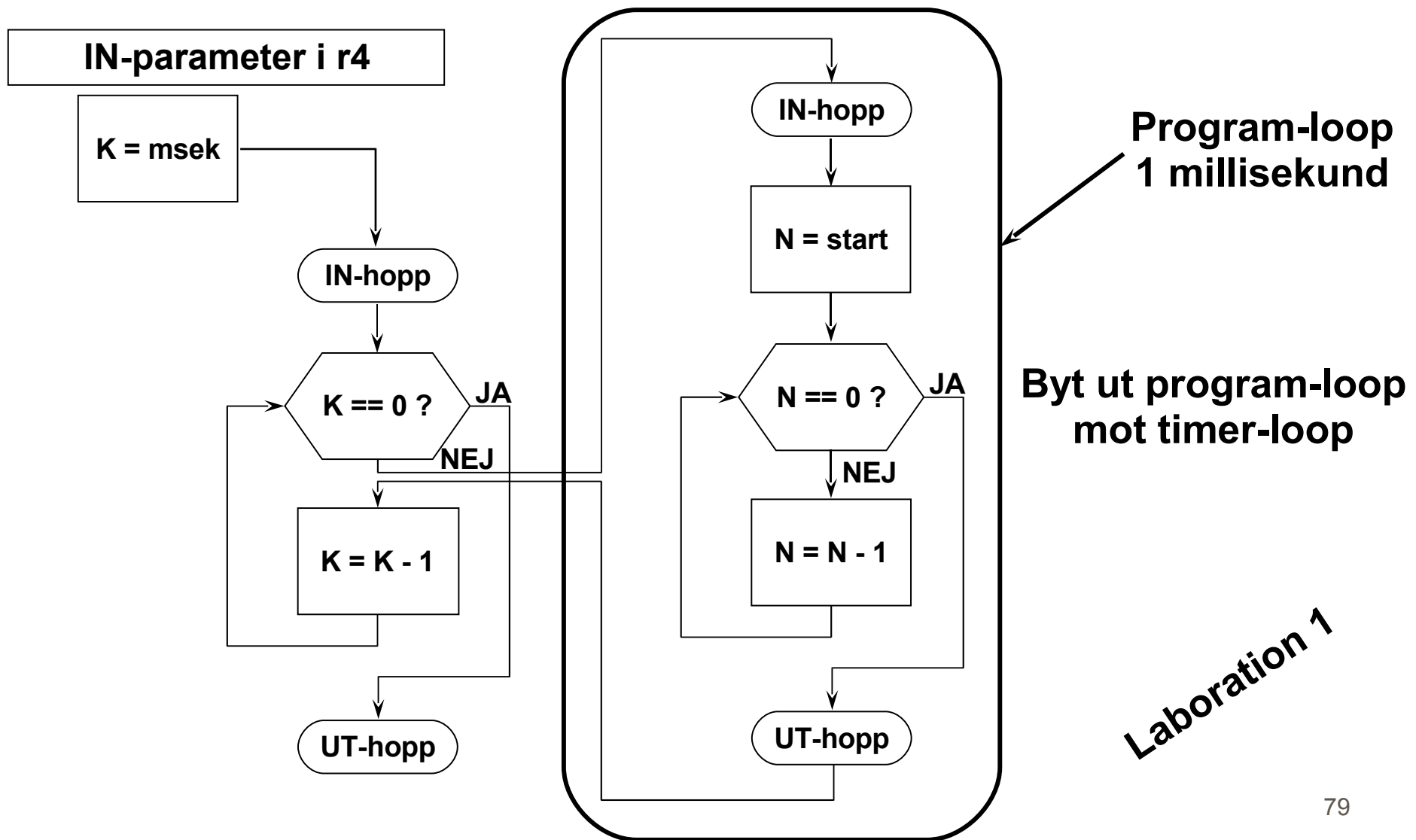
Flödesschema för subrutin som väntar på Time-Out



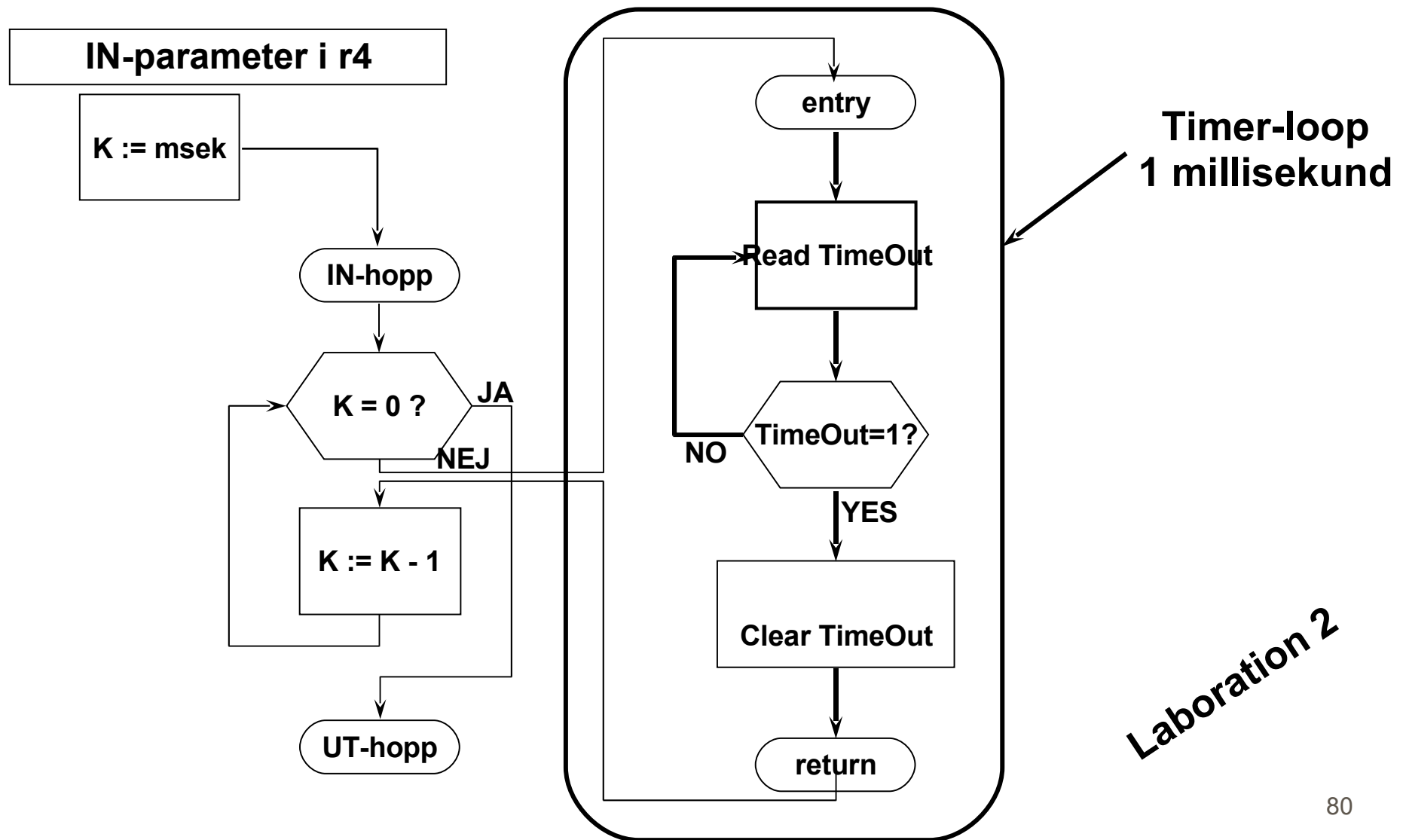
**Wait for
READY**

**Action
&
Acknowledge !**

Flödesschema DELAY med program-loop



Flödesschema DELAY med timer



Timer – räknare som räknar ned

- Räknaren laddas med önskad periodtid
 - tiden väljs med program
- Räknarinnehållet minskas med 1 varje klockcykel
- När räknaren kommit ner till noll så ettställs Time-Out
- Räknaren kan laddas om av hårdvaran (med samma värde igen) vid Time-Out

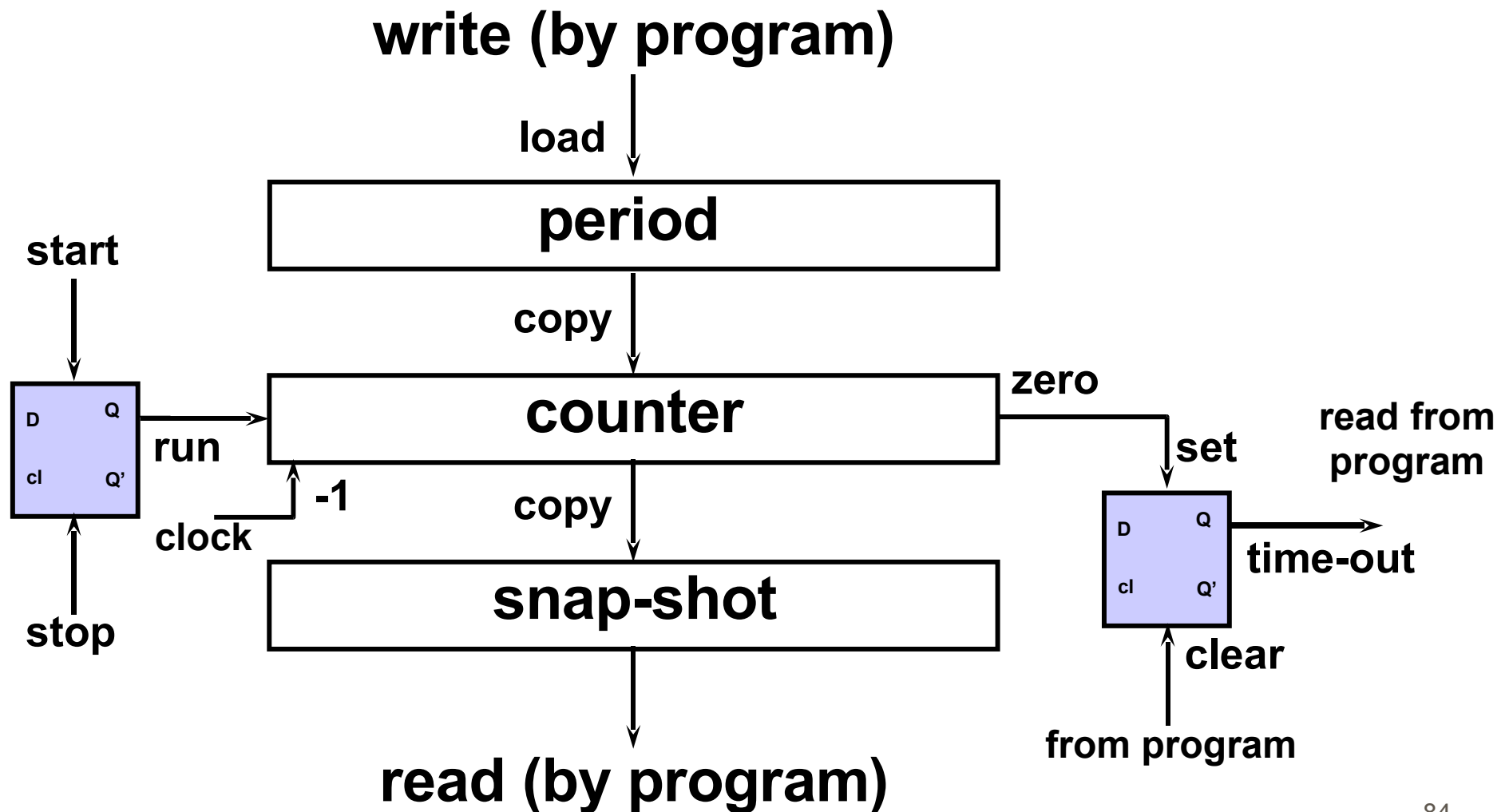
Timer – statusvippta Time-Out

- Time-Out kan läsas med program
- Time-Out finns också tillgänglig som elektrisk signal
- **ettställs** vid Time-Out
- ska **nollställas** av program

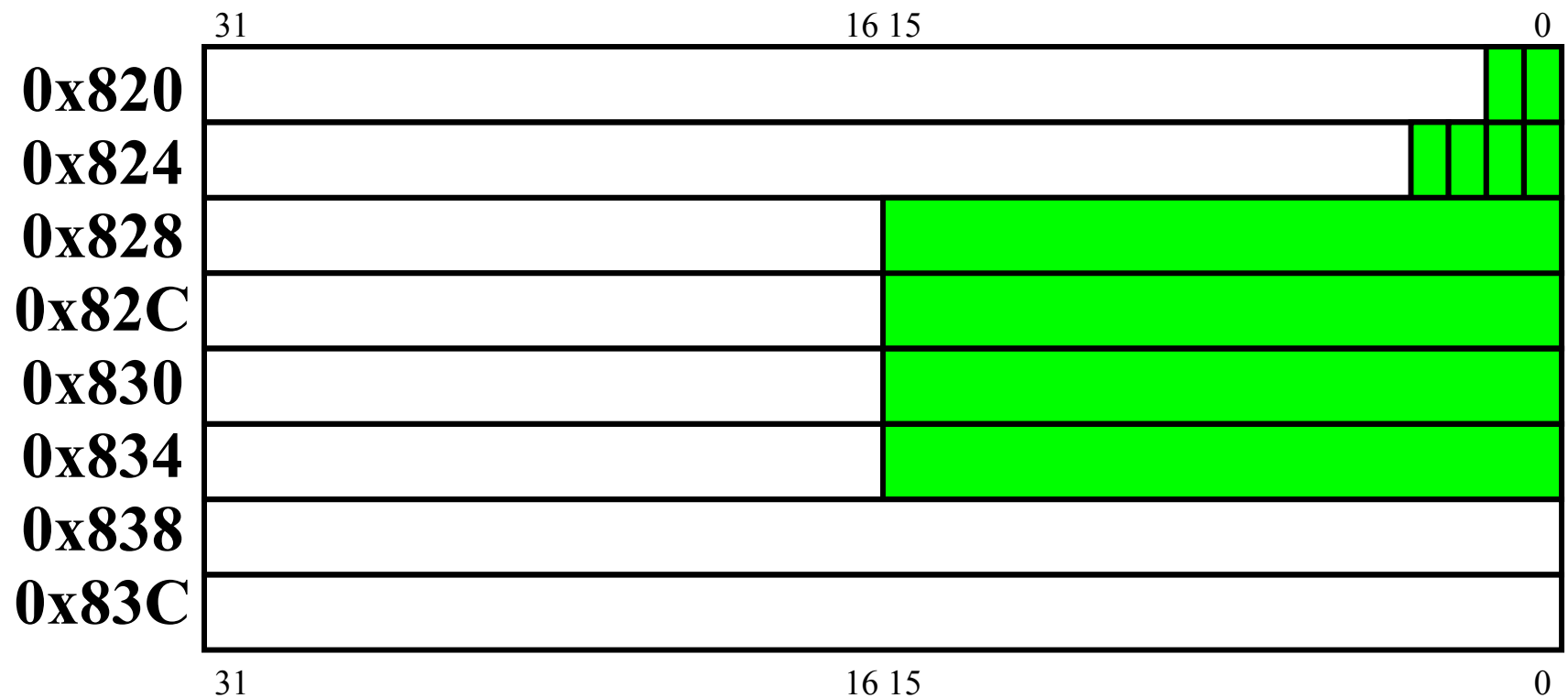
Timer – statusvipppa Run

- Run ettställs av program
 - annars startar inte timern
- Kan nollställas av program

Timer – skiss på hårdvaran



Nios2 Timer Memory-mapped



Timerns register är halvbreda

- Alla Timer-register kan läsas och skrivas med LDW (Load Word, 32 bitar)
- Bara de 16 bitarna till höger används

Nios2 Timer

0x820 status

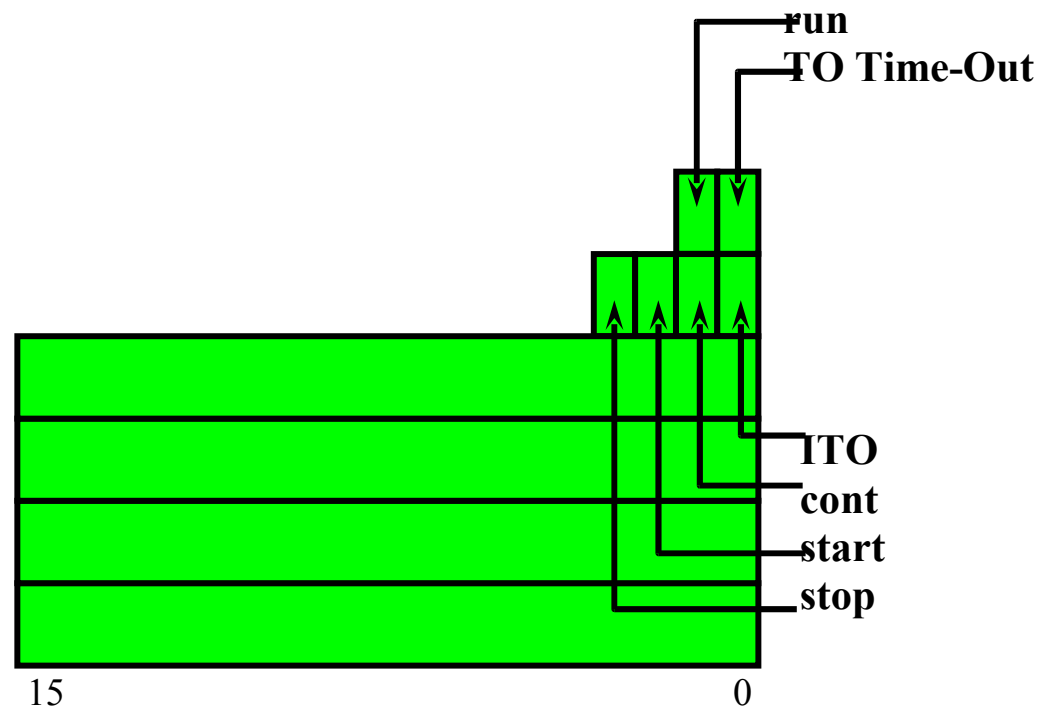
0x824 control

0x828 periodl

0x82C periodh

0x830 snapl

0x834 snaph



Nios2 Timer

- Time-Out = TO, bit 0 på adress 0x820
- Run = bit 1 på adress 0x820
- Återställning av Time-Out:
skriv **valfritt** värde till 0x820
 - värdet ignoreras, men bussöverföringen (skrivning till 0x820)
gör att Timern återställer Time-Out
- Start = kör igång, ettställ bit 2 på adress 0x824
- Stop = stoppa, ettställ bit 3 på adress 0x824

Minneskarta Nios2

Alla adresser är hexadecimala

- 0 - 7FF boot_monitor_rom
- 810-81F led_pio
- 820-83F timer_0
- 840-84F button_pio
- 850-85F dip-pio
- 860-86F uart_0 ("hyperterminal")
- 880-88F uart_1 ("annat labkort")
- 20000-3FFFF sram, 256 kByte